

QCon
全球软件开发大会

有道龙虾LobsterAI的养成 与实践

刘刚

网易有道



极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 6月 📍 上海

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：6月26-27日

- AI Infra 系统工程
- 多 Agent 协作与实践
- 多模态融合
- 模型训练与推理创新
- 数据平台与特征服务

🕒 8月 📍 深圳

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海

👥 1200人

QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全

目录

- 01 有道龙虾产品全景
- 02 有道龙虾的起源
- 03 架构选型与设计实践
- 04 Vibe Coding实践
- 05 未来规划

01

有道龙虾产品全景

全场景个人助理 Agent



工具碎片化的困境

每天在 5-10 个工具间反复切换

- 写文档 → 工具 A
- 做表格 → 工具 B
- 搜信息 → 工具 C
- 发邮件 → 工具 D
- 做 PPT → 工具 E

更深层的问题

工具不互通 — 搜完信息还得手动粘贴

无法自动化 — 重复性工作每天手动执行

不记得你 — 每次从零开始

只能坐在电脑前 — 离开就无法操控

什么是有道龙虾



打造 PC 端全场景个人助理 Agent，深度覆盖数据分析、文档处理、PPT 生成、视频制作、邮件 workflow 等日常生产力场景。

全场景Agent助理



告别繁琐的命令行配置。作为一款标准的 PC 桌面端应用，双击即可完成安装，为用户提供零门槛、沉浸式的 AI 交互体验。

极简安装开箱即用



国内大厂首个全面开源的 Agent 级产品。2026 年 2 月上线首周 GitHub Star 突破 3000，与全球开发者共建开放的 AI 生态。

100%开源龙虾

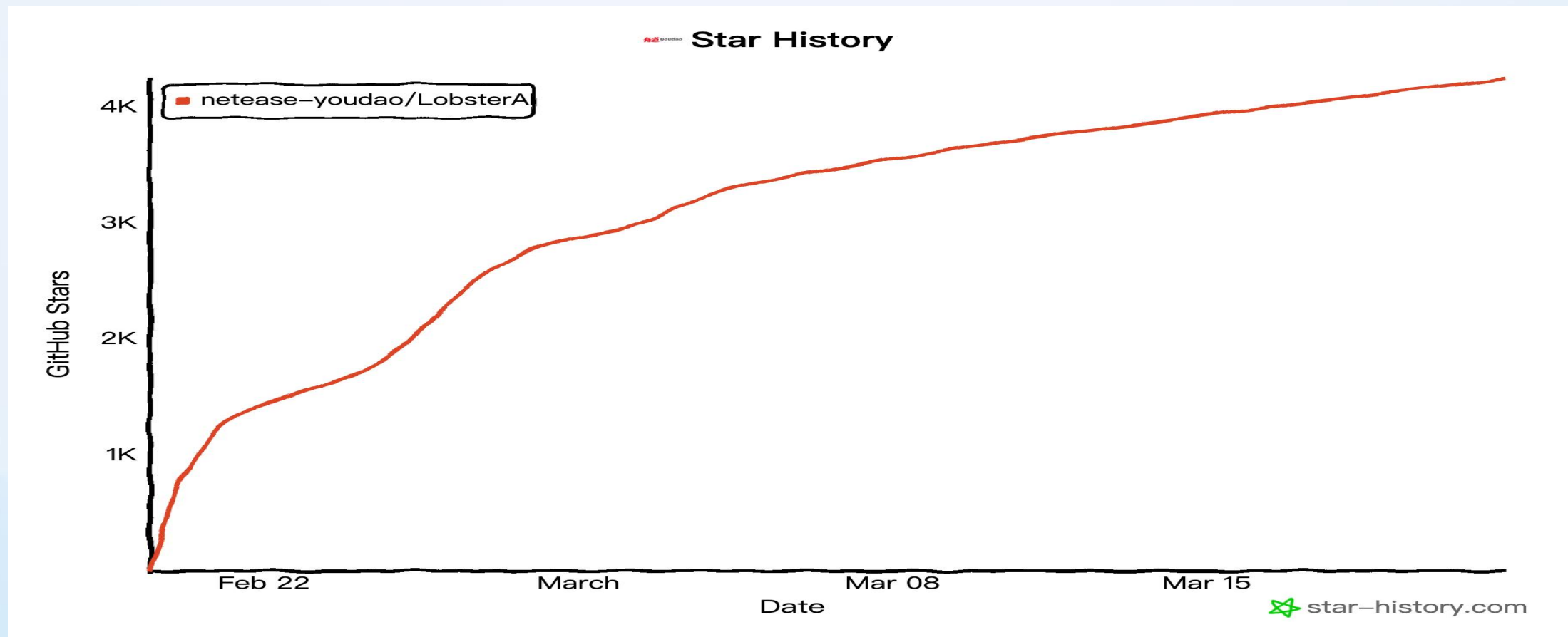


告别全量文件云端上传，支持本地构建知识库与代码执行。完美兼容本地私有化大模型，让企业敏感数据真正留在本地。

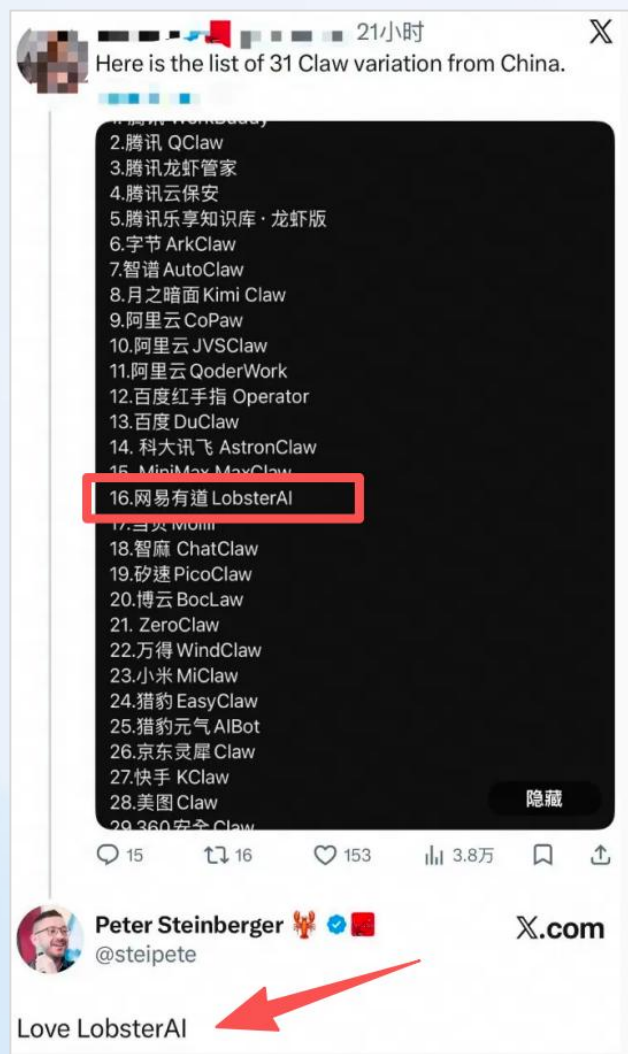
数据隐私安全可控



开源开放的AI生态



OpenClaw 创始人公开称赞





有道龙虾 — 全场景 Agent 助理

一句话驱动

"帮我分析数据，生成报告，发到邮箱"

技能

文档/表格/PPT/视频/搜索/邮件

定时任务

"每天 9 点收集科技新闻"

手机远程操控

钉钉/飞书/Telegram/Discord

越用越懂你

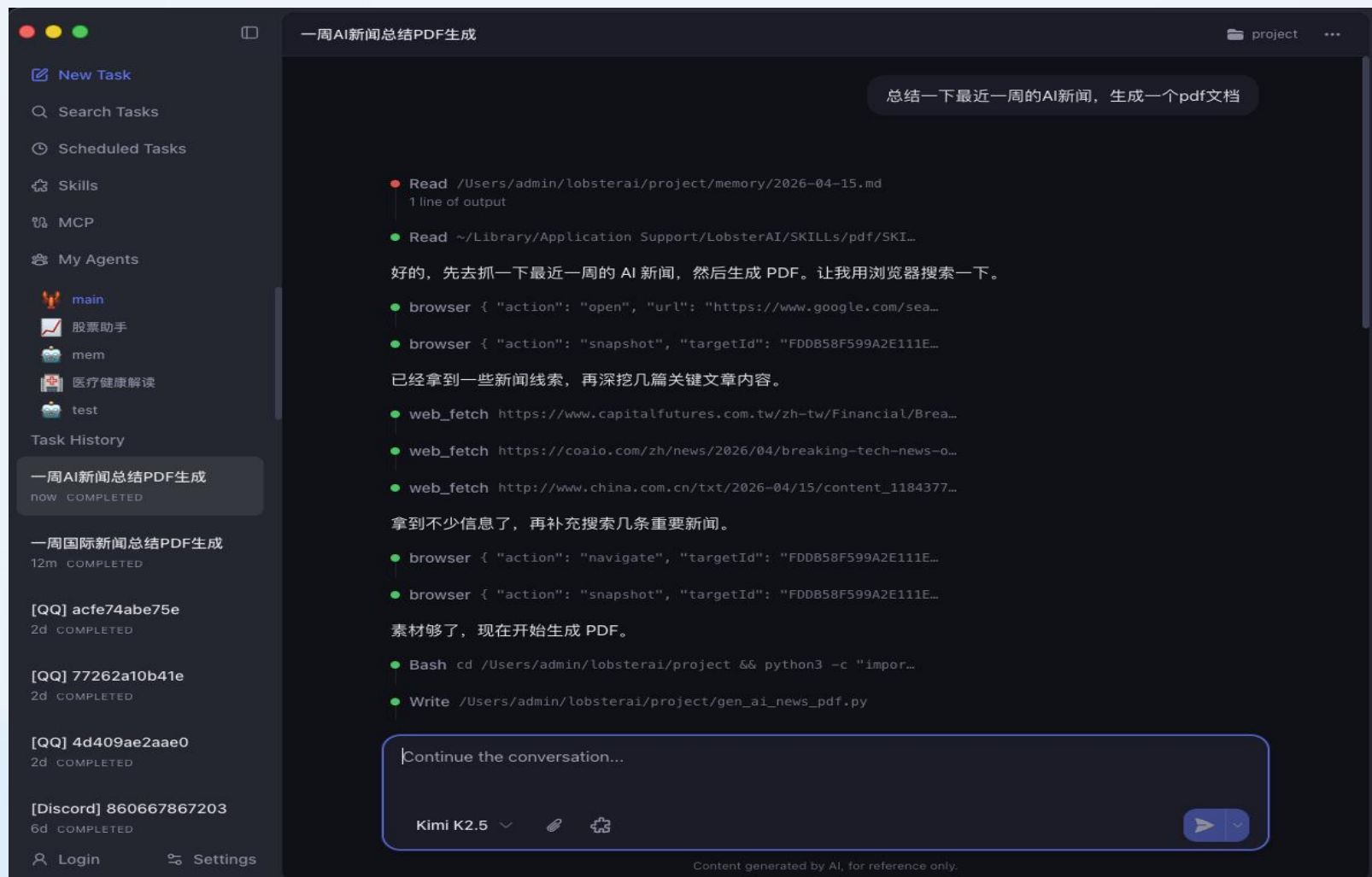
记忆系统，跨会话记住偏好

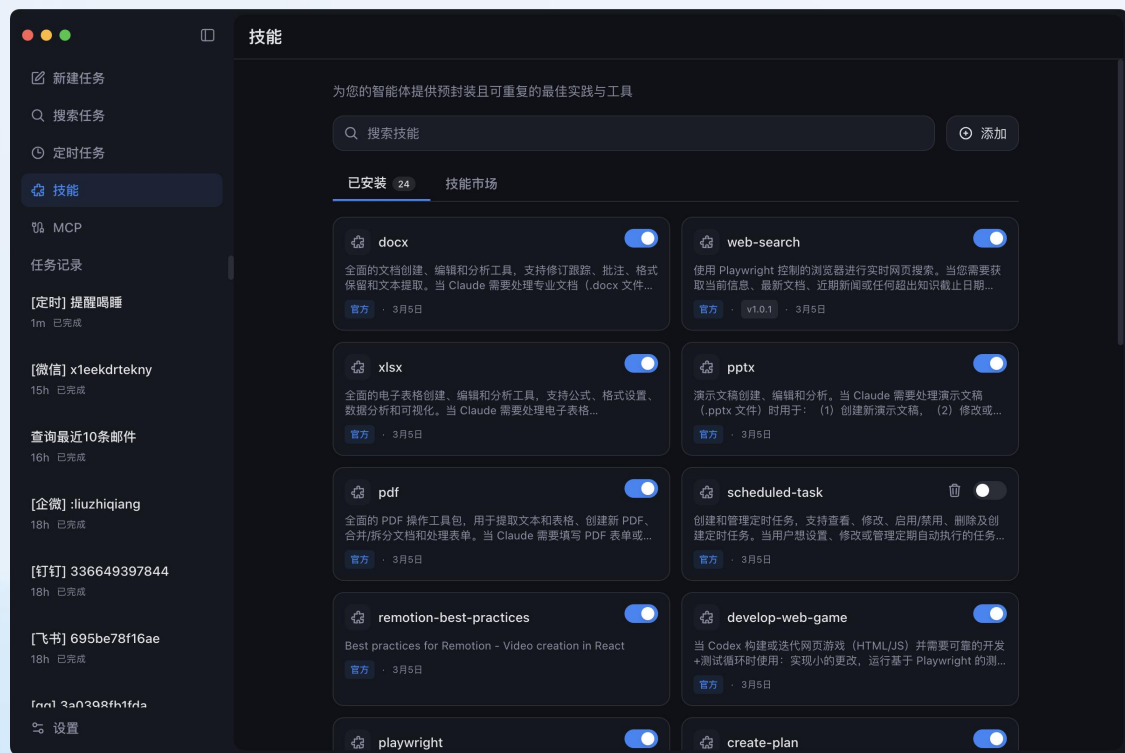
跨平台

macOS / Windows / Linux



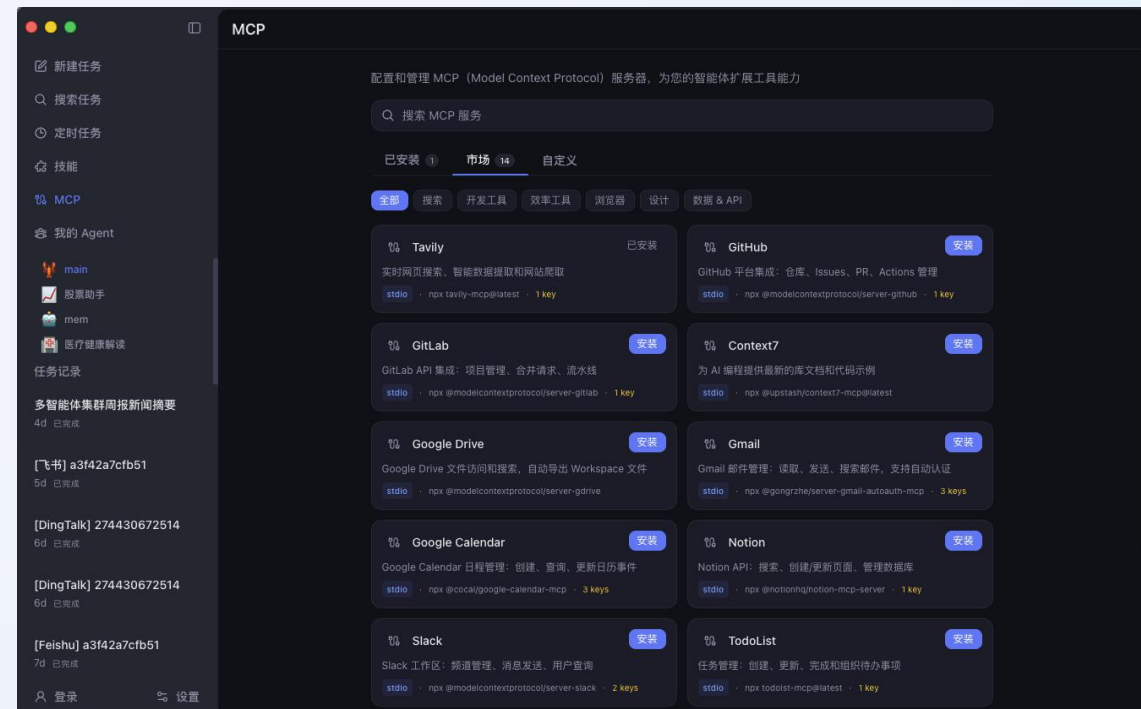
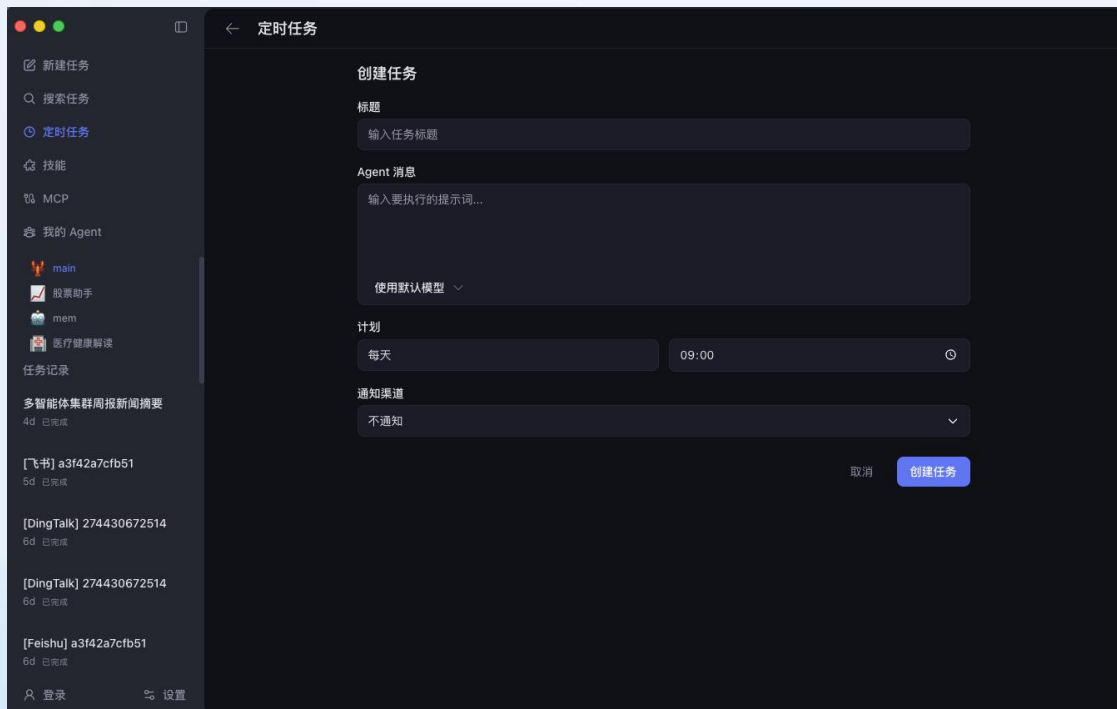
有道龙虾-功能概览





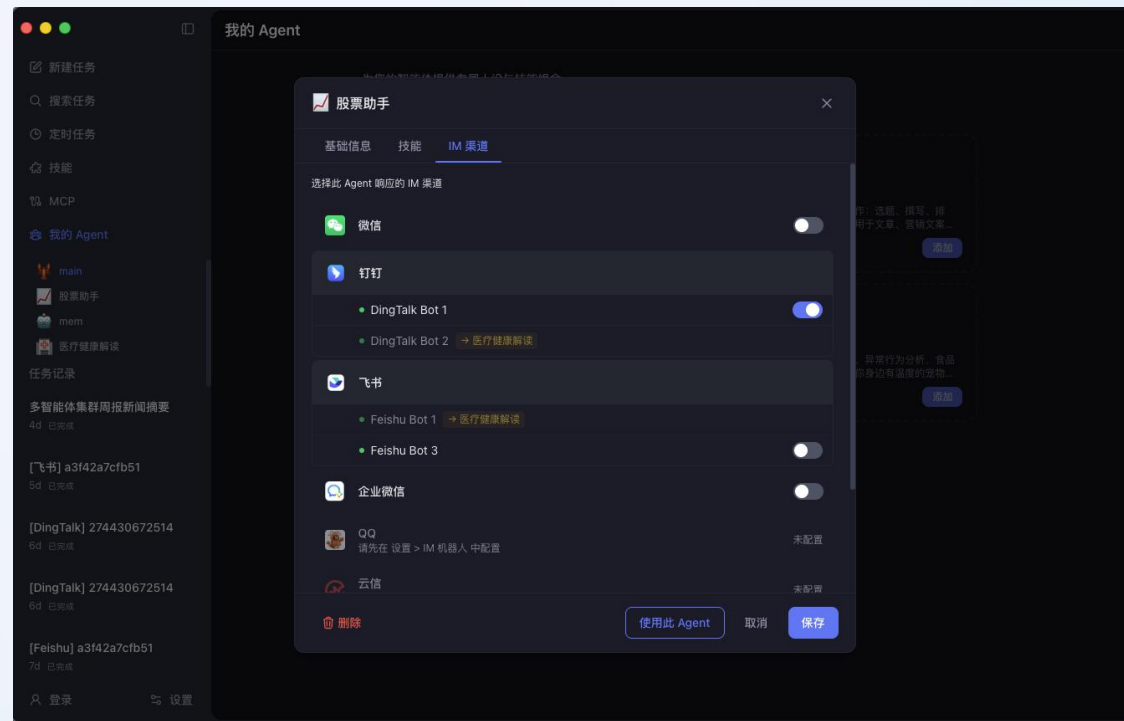
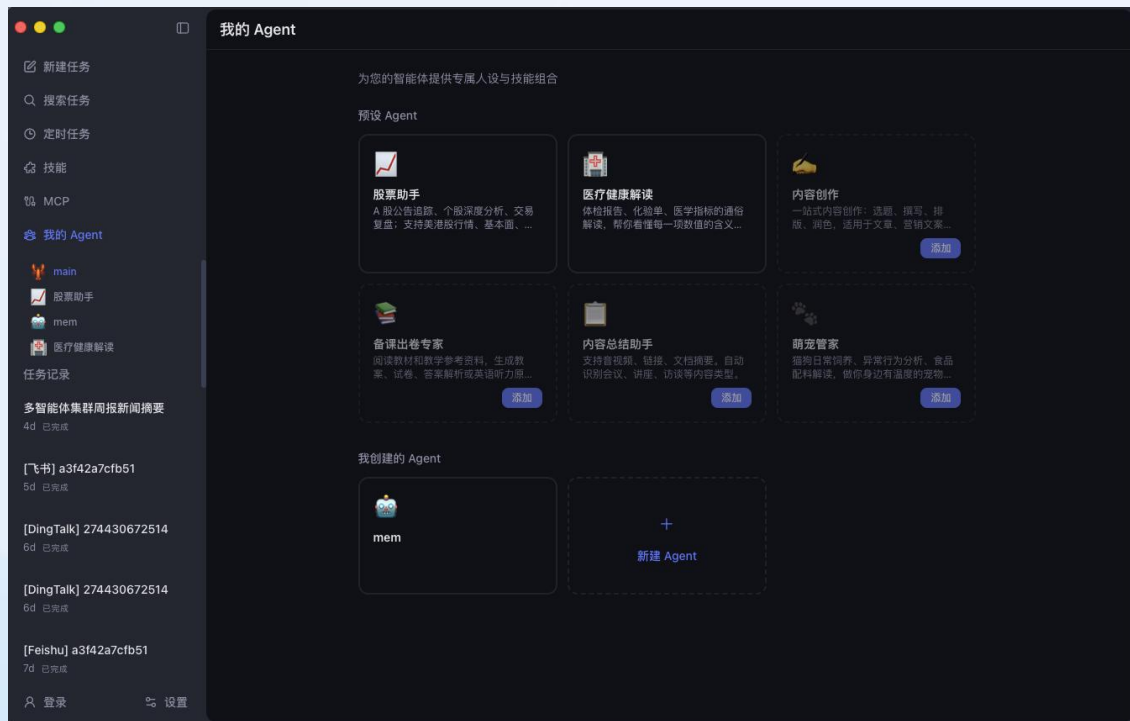


有道龙虾-定时任务和MCP





有道龙虾-多Agent和多机器人



有道龙虾-核心功能



只需要输入最终目标（比如“帮我分析这份财报并做成 PPT”），它会自己思考第一步干什么、第二步干什么，遇到问题自主调用工具解决。

Agentic 架构



内置丰富 Skills 技能，自动执行复杂工作流；深度支持 MCP 协议，一键打通本地与企业外部数据孤岛。

Skills & MCP



支持自然语言设定灵活周期任务，后台独立静默运行互不干扰，完成后自动将结果推送到指定频道，实现 7×24 小时工作流闭环。

定时任务



全面打通钉钉、飞书、企微等主流 IM。一部手机即可随时远程唤醒 Agent 派发任务，不在电脑前，也能用手机微信/钉钉指挥它干活。

手机端控制



使用案例

场景 1

每天需要追踪行业动态

每天早上自动获取最新资讯，
无需手动操作。

用户输入

USER

"每天早上 9 点，搜索 AI 领域最新新闻，生成一份 Word 日报，发到我邮箱"

最终产出



全程无人值守，第二天打开邮箱就能看到

执行流程

1

定时任务触发

scheduled-task

2

搜索并抓取新闻

web-search

3

生成 .docx 日报

docx

4

发送邮件

imap-smtp-email

使用案例

场景 2

出差路上临时需要处理任务

手机上发一句话，桌面 Agent 自动完成复杂任务。

用户输入

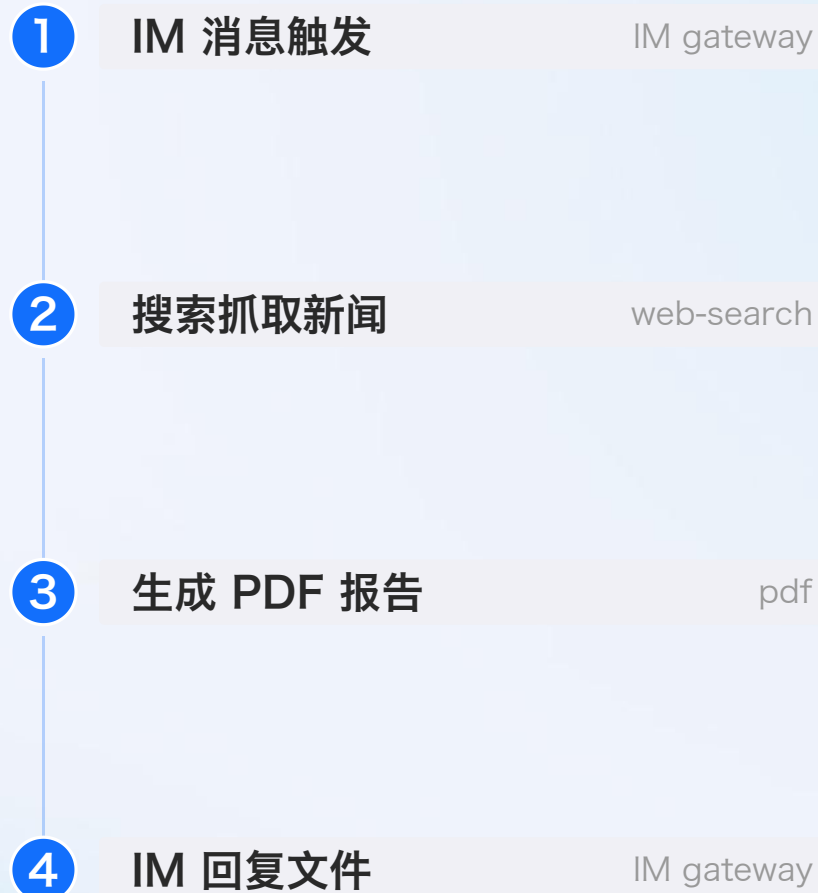
USER

"基于电脑上 XX 的资料，结合竞品 XX 最近的融资新闻，整理成一页 PDF 发给我"

最终产出

✓ 手机上发一句话，桌面 Agent 自动完成

执行流程



使用案例

场景 3

销售/运营每天下载报表 进行数据分析

从数据下载到分析再到发送，
Agent 一条龙搞定。

用户输入

USER

"帮我下载 XX 平台昨天的销售数据，根据要求 xxx，
生成一份数据报表，发送到 xxx 邮箱"

最终产出



从数据下载到数据分析再到邮件发送，Agent
一条龙搞定

执行流程

1

数据下载

2

匹配用户需求

3

生成数据报表

4

发送到指定邮箱

imap-smtp-email

场景 4

HR 自动简历筛选

自动化筛选简历，HR 只需最终审阅微调。

用户输入

USER

"帮我把 XX 网站上近 7 天未筛选的简历，按如下要求筛选出来，具体要求为：XX"

最终产出

✓ 简历筛选自动完成，用户只需审阅微调

执行流程

- 1 抓取招聘网站简历
- 2 按要求自动筛选
- 3 生成筛选报告
- 4 发送完成提醒

Claw类产品对比

维度	OpenClaw	云端 Claw	其他桌面端 Claw	有道龙虾 (LobsterAI)
上手与交互体验	纯命令行操作, 需配置开发环境。	订阅逻辑, 卖云主机+模型 token	开箱即用, 双击安装。	开箱即用, 双击安装。
大模型自由度	CLI 配置或手动修改底层配置文件	一般绑定自家模型	深度绑定自家模型, 送免费 token, 用完卖订阅服务	自由配置 DeepSeek/Kimi 等全球顶尖模型, 并完美兼容 Ollama 本地私有模型。
安全性	开源	业务报表和敏感文档必须全量上传至云端, 极易造成企业机密外泄。	必须登录才可用, 隐私不可控 重度依赖云端 API 交互, 核心业务数据依然需要“出网”。	开源, 完全免费, 不上传任何用户私人数据和埋点, 仅上传给大模型必要的片段数据。
产品定位	个人助理	云端助理	桌面端助理	桌面端助理

02 有道龙虾的起源

从chat工具到全场景个人助理 Agent

有道龙虾的起源

团队：AI+教育

硬件产品

词典笔、答疑笔

小P老师

K12全科答疑

Chat模式

视频讲解

基于用户输入生成2分钟图文讲解视频
教育场景AI Agent



语文

讲解《茅屋为秋风所破歌》还原当时的场景

有道龙虾的起源

突破垂类Agent边界，布局通用场景

现有痛点

垂类Agent能力局限，仅能解决单一场景问题，普适性不足

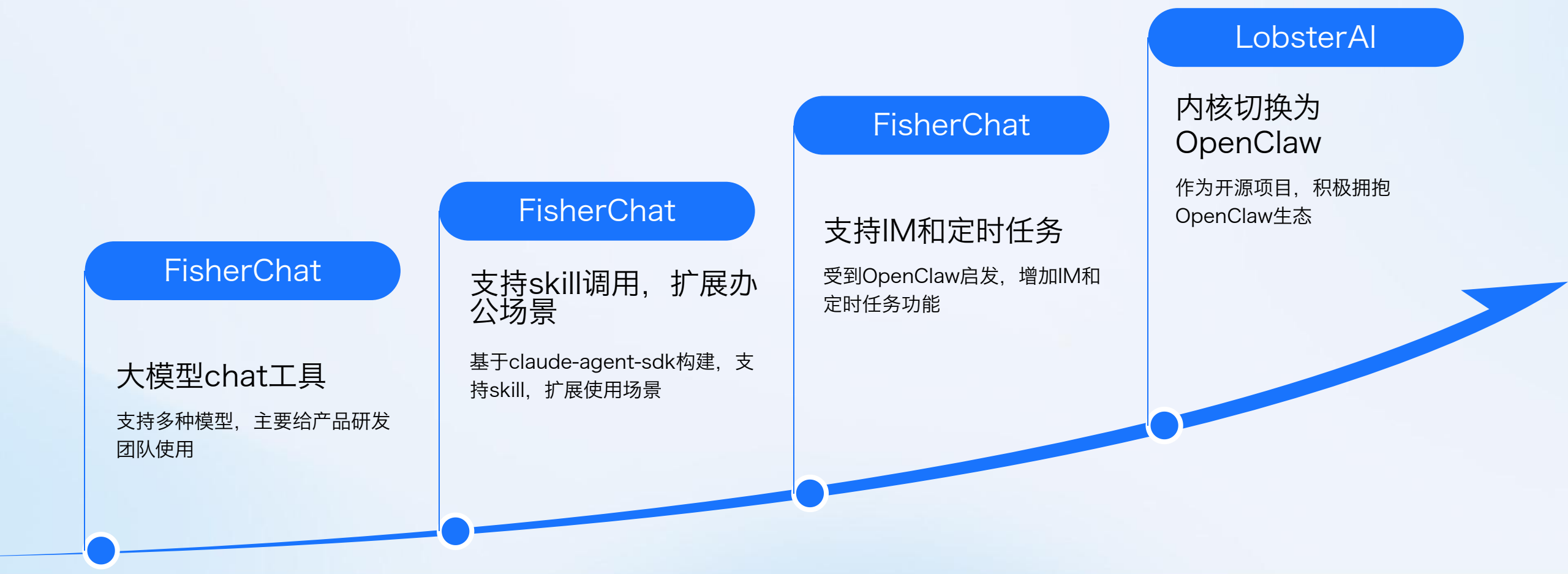
需求洞察

2025年AI编程Agent工具普及，非技术人员急需易用的AI办公工具

原型诞生

基于Claude Agent SDK，开发产品原型，适配普通用户交互逻辑

有道龙虾的演进



团队内部工具

Chat功能

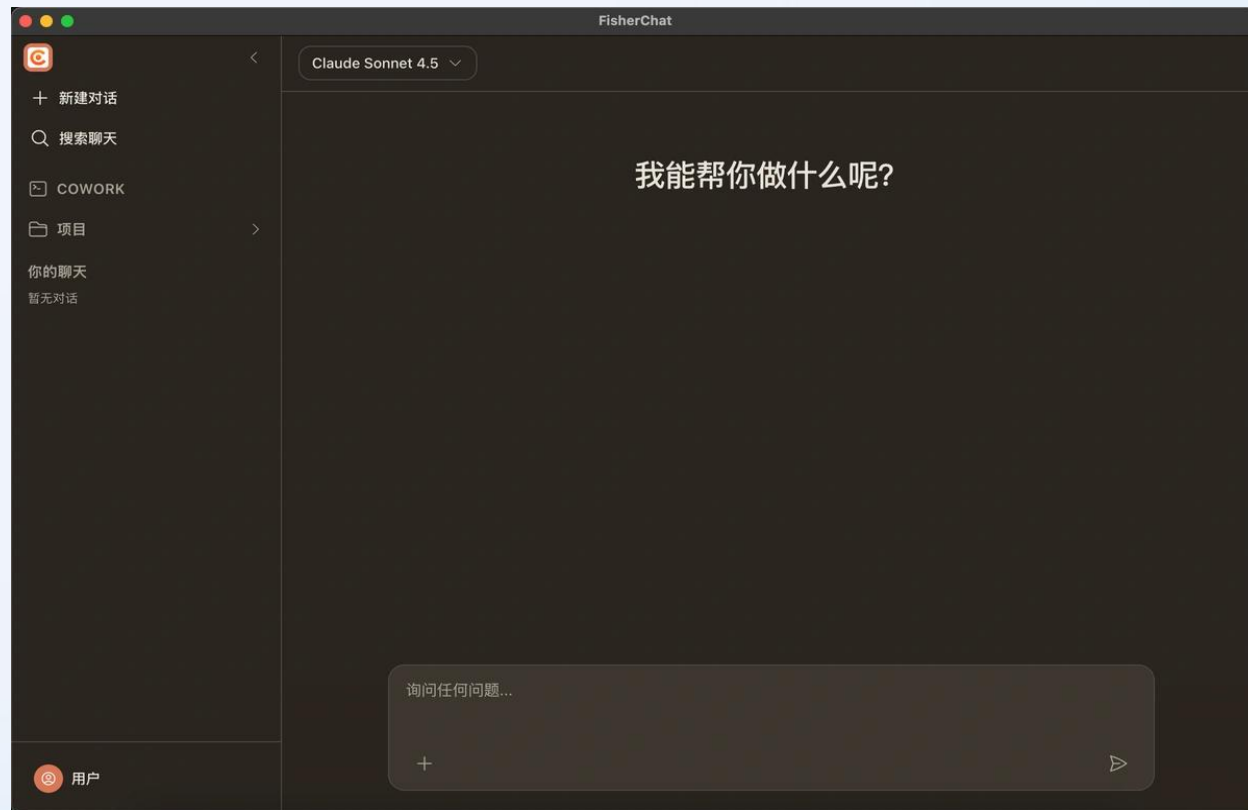
支持多种模型，内置支持 OpenAI（GPT 系列）、Anthropic（Claude 系列）、Google（Gemini 系列）、DeepSeek、Ollama（本地模型）等主流提供商

Skills功能

基于claude-agent-sdk构建，支持skill，扩展使用场景

项目管理

支持为不同使用场景创建独立的 Project，配置专属 System Prompt
指定工作目录，让 AI 在明确上下文中工作
多会话管理，历史记录持久化存储



开源桌面Agent

定时任务

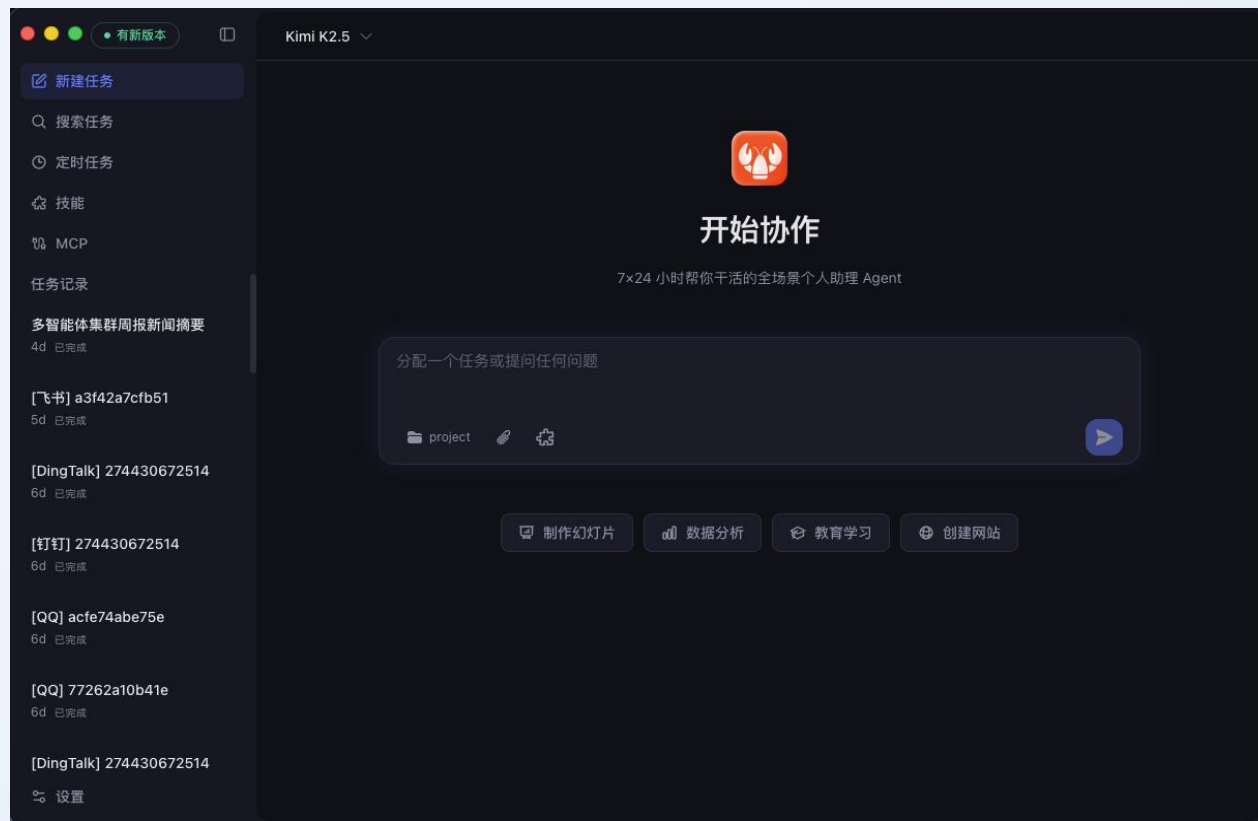
支持通过桌面端软件，IM 机器人设定定时任务

MCP

支持MCP功能，允许用户安装第三方MCP工具

IM

支持IM软件：钉钉，飞书，Telegram和Discord



有道龙虾

OpenClaw内核

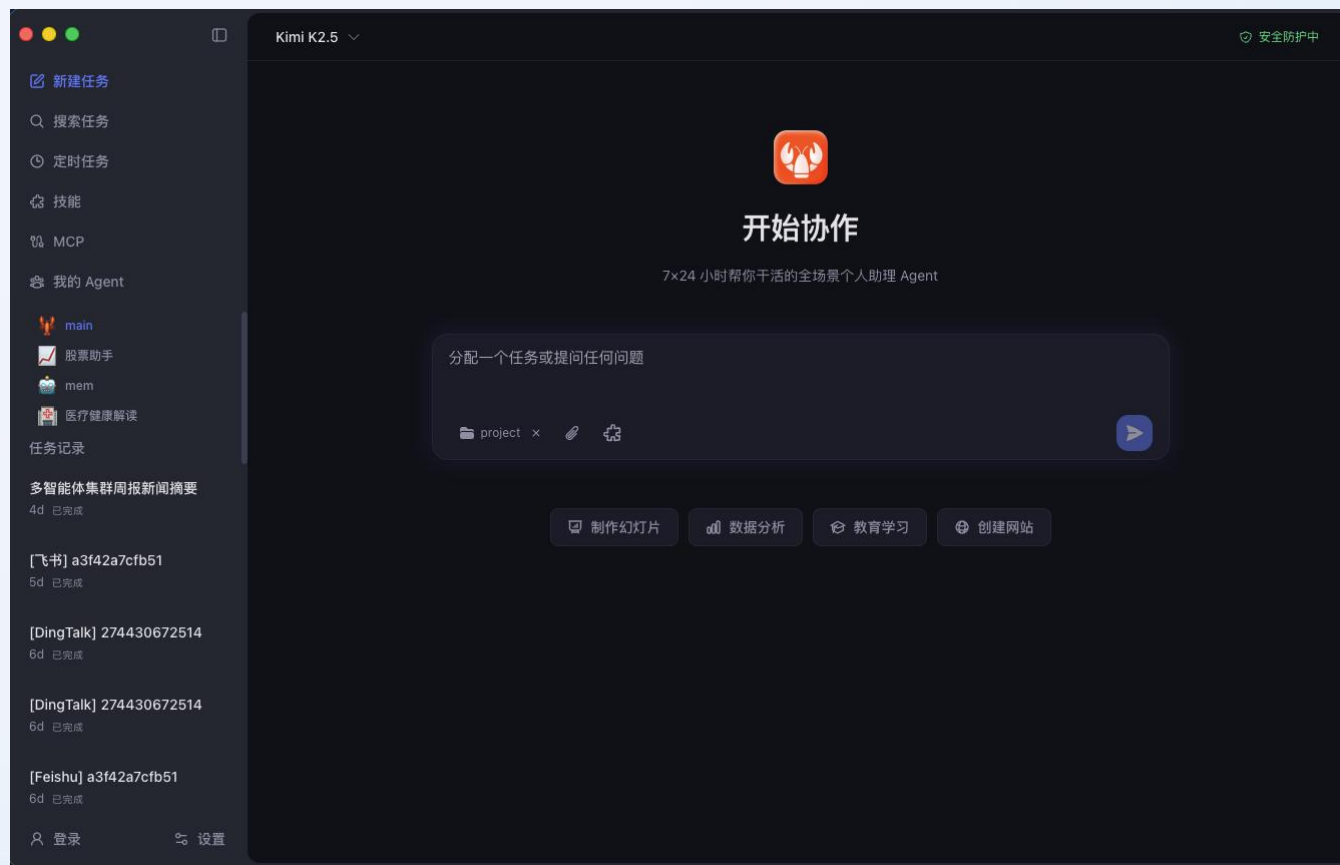
内核从claude-agent-sdk切换为Openclaw运行时

多Agent

支持配置多个agent, 提升不同场景的使用效率

IM多机器人

飞书, 钉钉, QQ等支持多机器人配置



03

架构选型与设计实践

有道龙虾技术框架

Electron框架

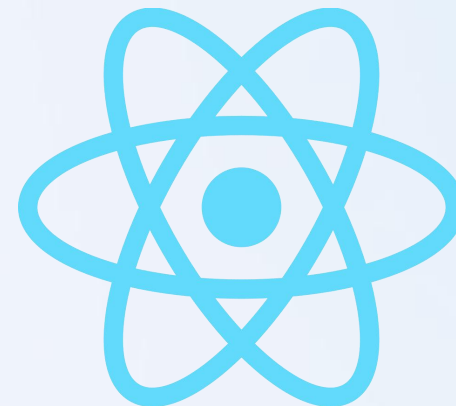
跨平台，支持windows， mac和linux
模型支持友好

React框架

React框架实现UI层渲染
模型支持友好

Typescript

静态类型，有利于构建复杂项目
模型支持友好





有道龙虾架构-claude-agent-sdk内核

整体架构

anthropic格式

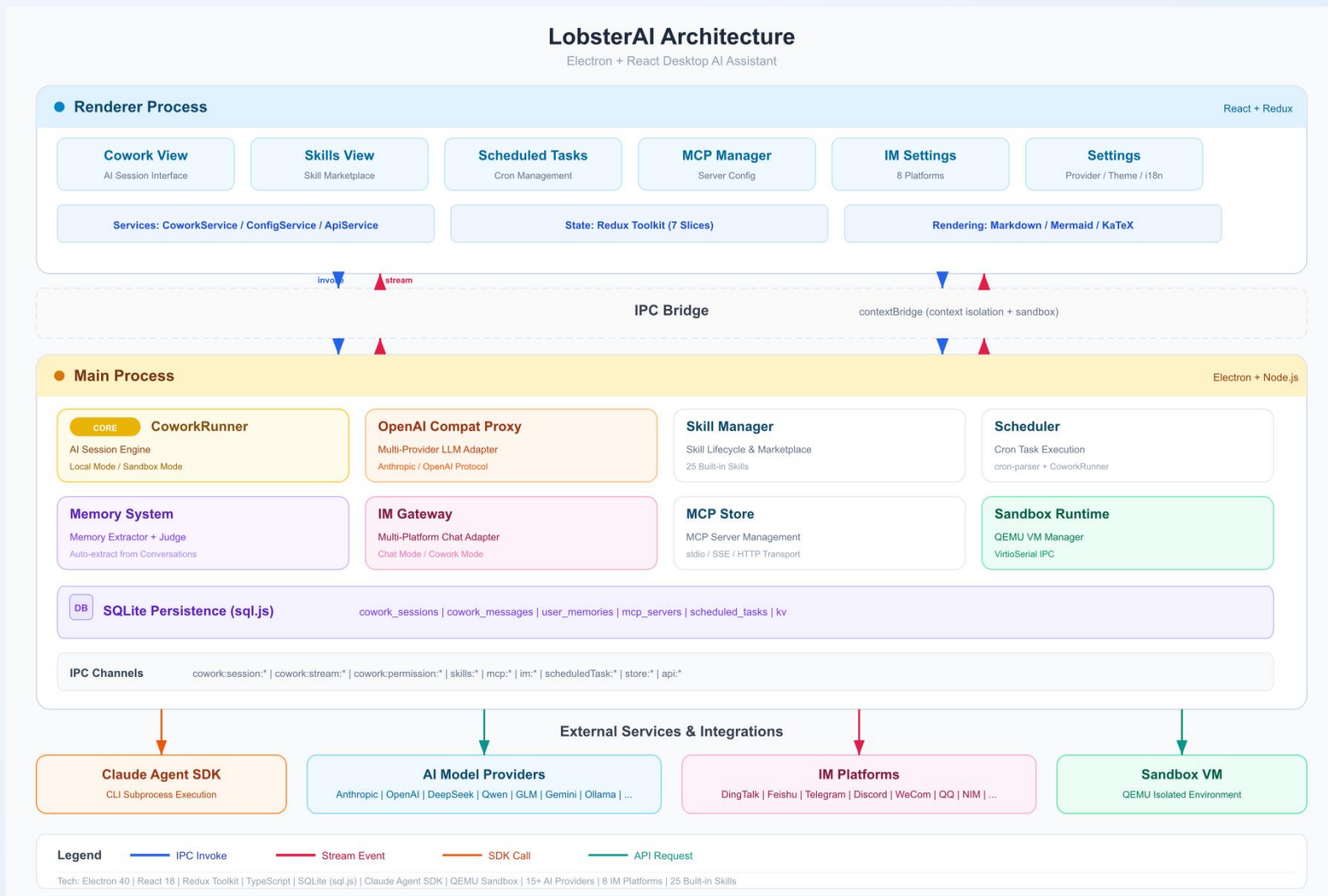
第三方模型需要转换为anthropic格式

IM 机器人

基于飞书，钉钉，Telegram等平台开放API实现连接，状态，富媒体信息需要处理

沙箱

Qemu+自定义镜像
windows运行较慢，需开启硬件加速



一个开源的、自托管的AI Agent执行框架

从“对话”到“执行”

不只是像 ChatGPT 那样聊天，而是能接管键盘、鼠标、浏览器和终端执行任务。

全天候自主运行

支持 7×24 小时在线，甚至在你睡觉时也能自动处理邮件、监控数据或编写代码。

强大的连接性

原生支持 WhatsApp、Discord、Slack等通讯工具。

持久化记忆

具备基于 Markdown 和上下文压缩技术的长效存储能力，比普通聊天机器人更懂你的习惯。



数据见证奇迹——OpenClaw 的现象级爆发

垂直式爆发

2026 年 2 月起，GitHub Star 曲线呈 90 度垂直上升，被社区称为“龙虾大爆炸”。

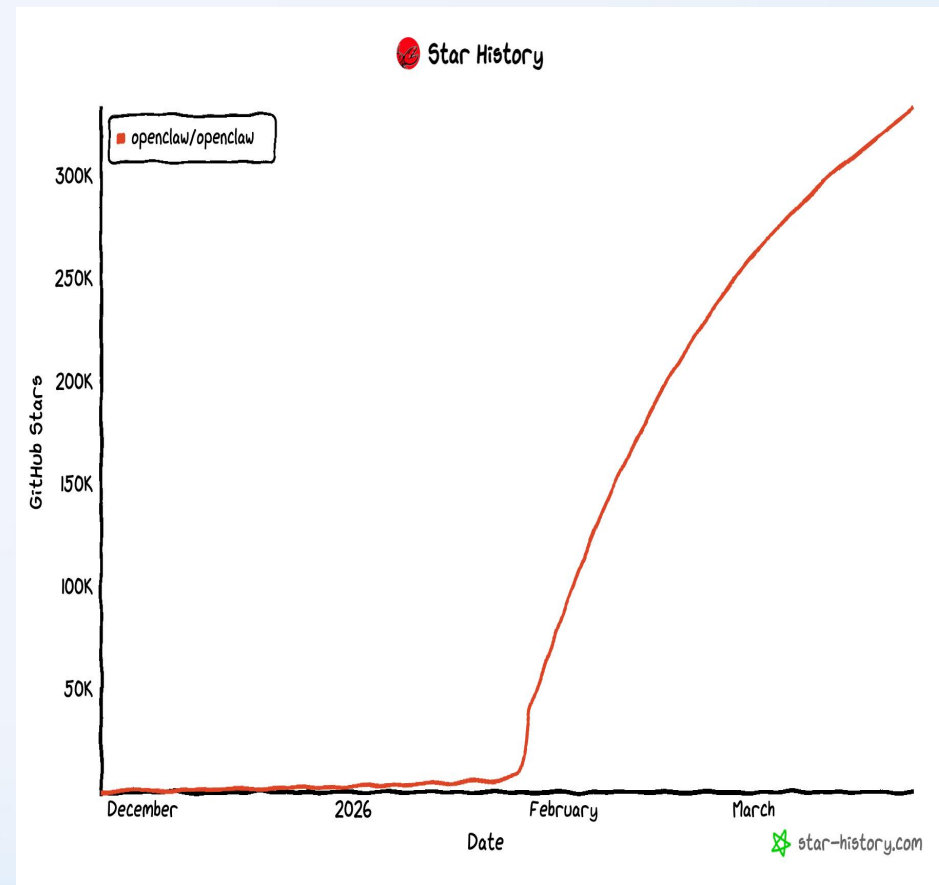
星标里程碑

72 小时：狂揽 100,000+ Stars（创 GitHub 历史最快纪录）。

60 天：突破 300,000 Stars，正式超越 React（10 年积累）和 Linux，登顶 GitHub 实用软件榜首。

流量冲击

官网一周内录得 2,000,000+ 次独立访问。



OpenClaw内核的优势

生态成熟

第三方支持会更好，例如微信，QQ等

Skill 插件生态（ClawHub）：内置上万种开箱即用技能（办公、开发、运维等）

功能高度复用，避免重复造轮子

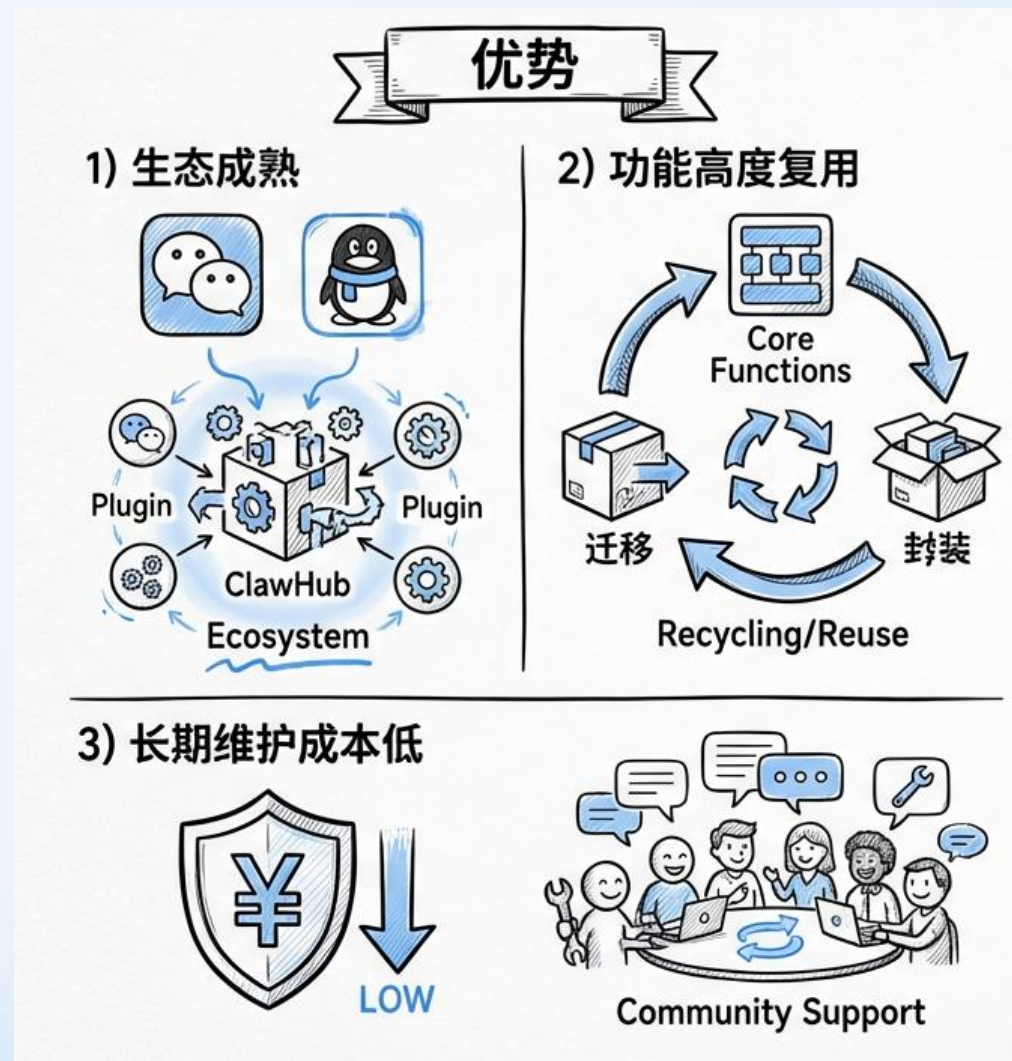
原有功能可快速迁移、封装、复用，不需要重新设计交互与调度。

新功能如多agent可直接复用

长期维护成本低

桌面端系统环境复杂，降低修复bug成本

社区活跃，问题修复、功能迭代更快



OpenClaw内核的劣势

桌面应用vs服务

LobsterAI是桌面应用，对交互响应速度要求较高
OpenClaw是个服务，一些配置变更必须重启网关，导致响应慢

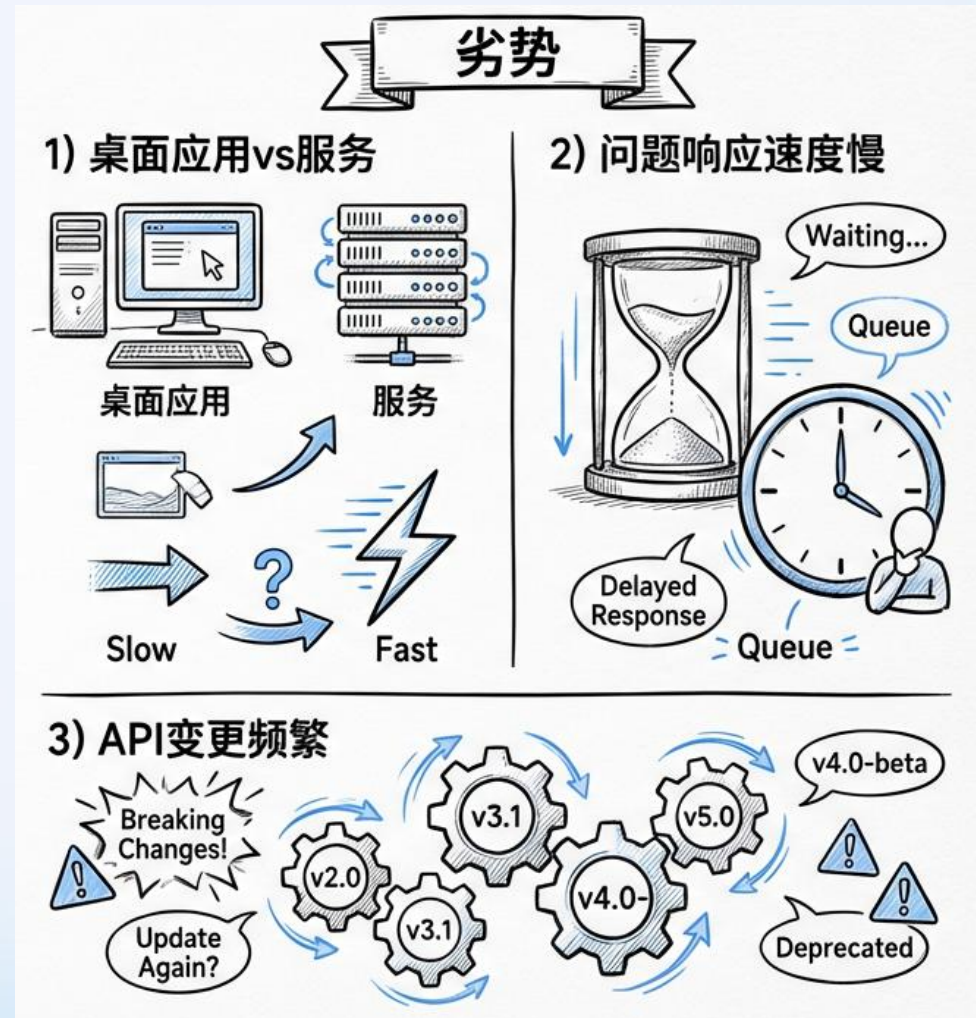
问题响应速度慢

部分功能如IM依赖第三方，bug修复需要等待
OpenClaw自身的一些bug，需要等待官方修复

OpenClaw问题

OpenClaw 目前处于极高速迭代期，API 变更频繁，需要投入人力进行版本对齐

OpenClaw功能多，依赖复杂，运行占用资源较高



OpenClaw内核架构

OpenClaw运行时

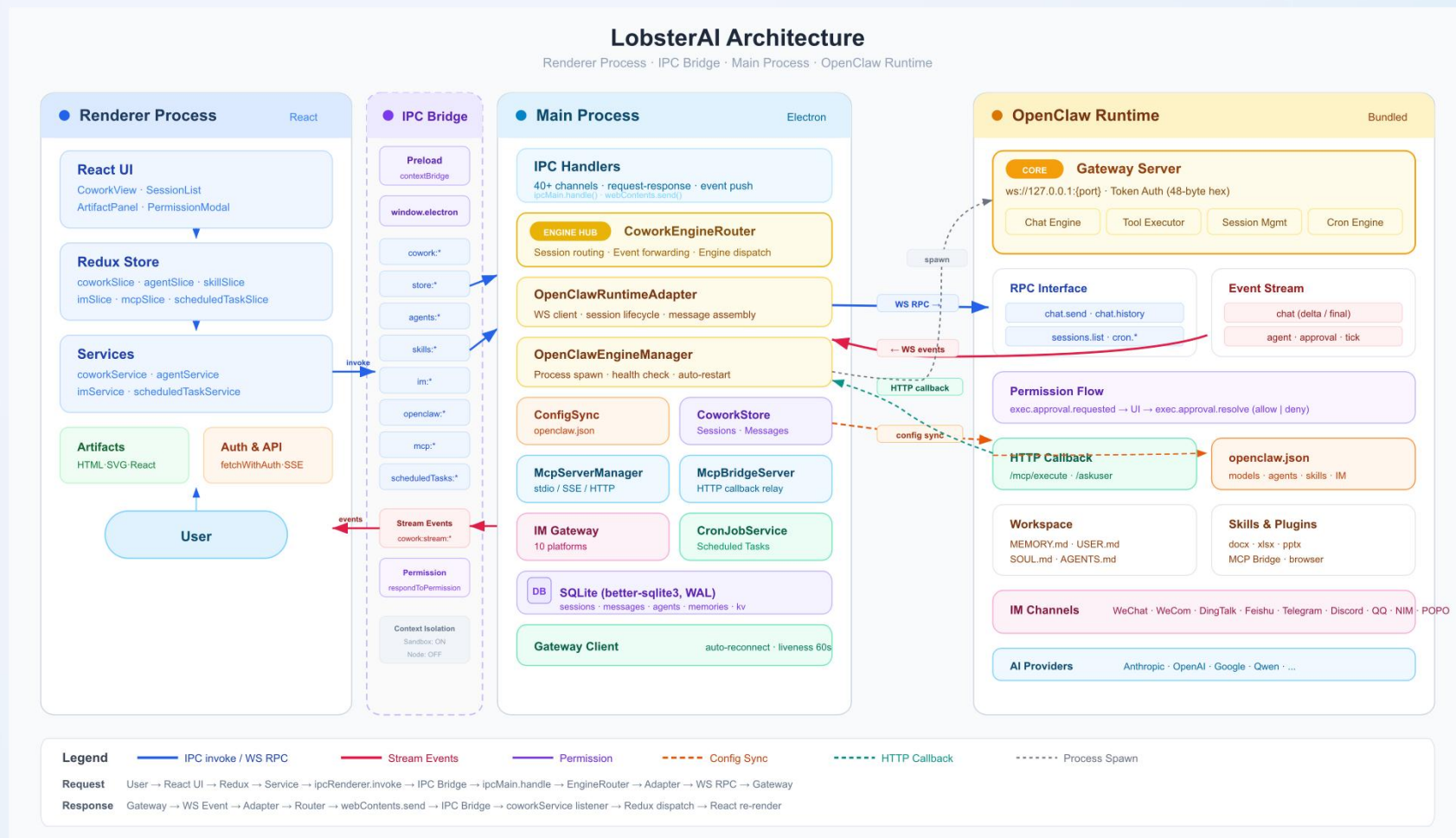
agent核心功能通过OpenClaw实现

IM机器人

通过OpenClaw的插件实现

MCP

MCP Bridge: 在 LobsterAI 侧启动 MCP server → 发现工具 → 转为 OpenClaw 的自定义工具注入



设计实践-OpenClaw

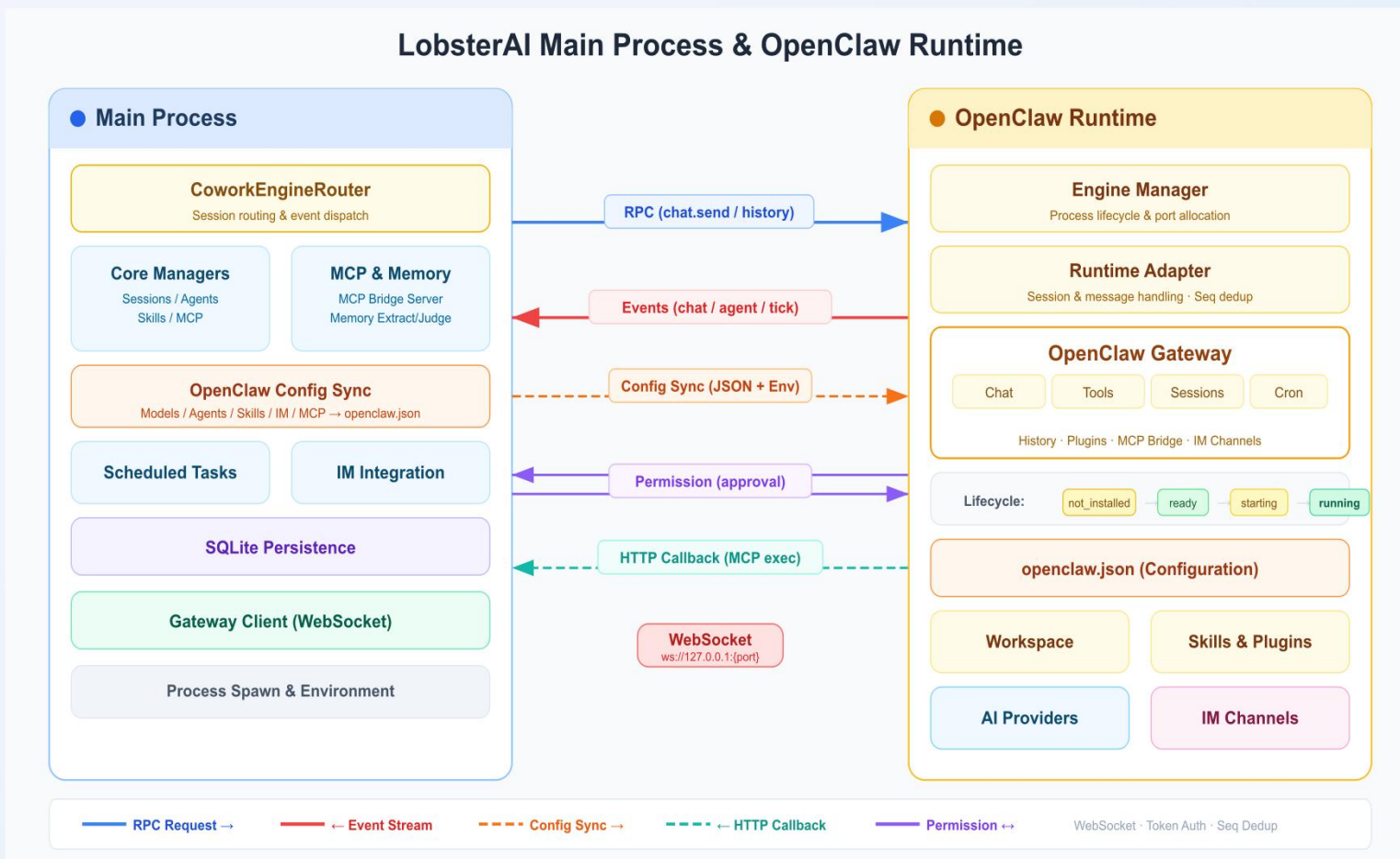
集成方案

源码集成

实现函数级别的功能调用，灵活性强
同一个进程，数据共享方便，功能延迟低
侵入性较强，OpenClaw升级成本较高

独立进程 (runtime)

只能通过OpenClaw提供的接口调用功能
OpenClaw runtime独立，OpenClaw升级成本较低
通过patch形式实现必要的功能修改（OpenClaw版本）
已有功能和OpenClaw独立存在，方便功能迭代和替换



设计实践-OpenClaw

runtime构建

源码管理

支持版本配置

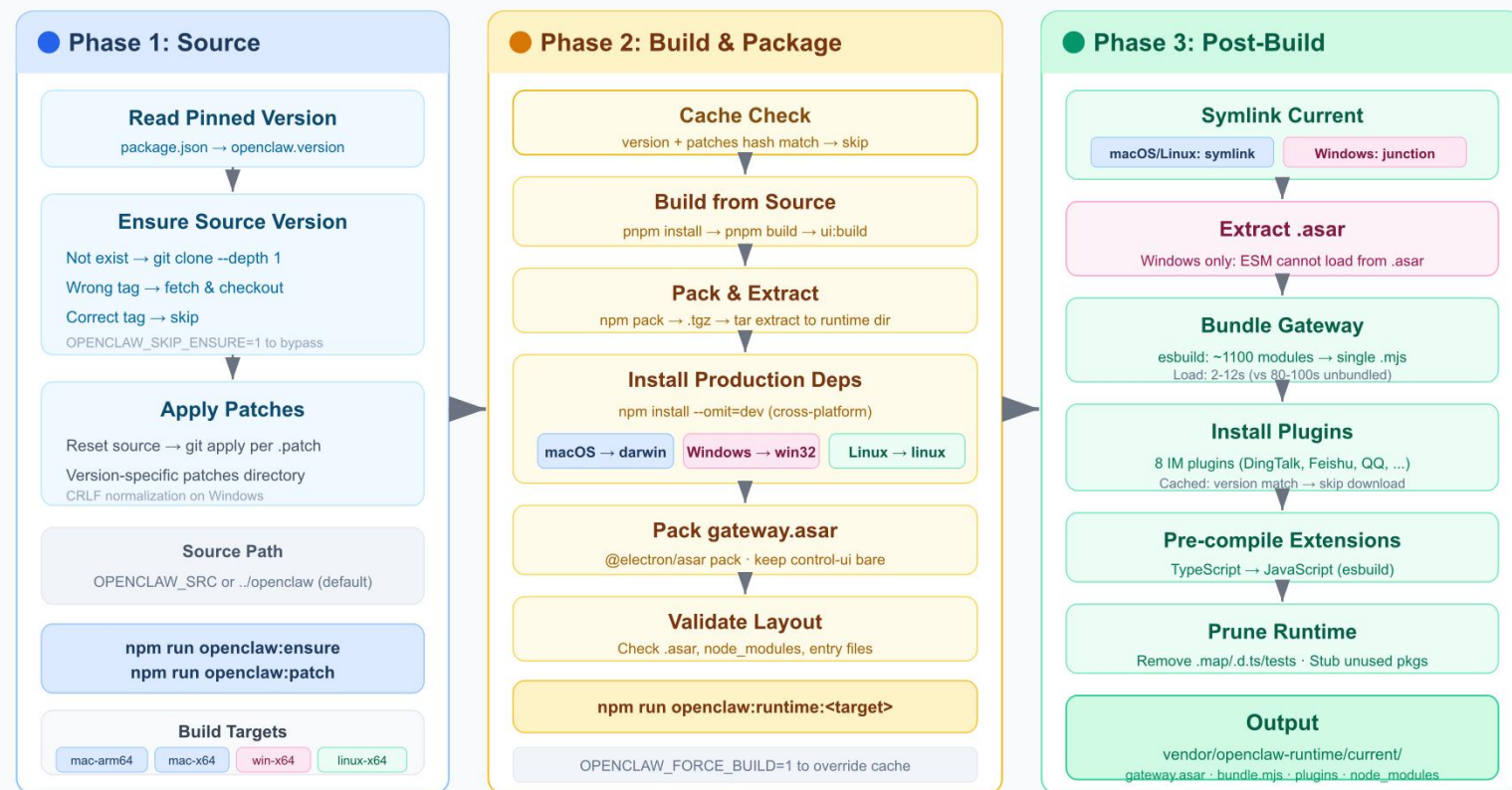
插件管理

OpenClaw插件配置

Patch管理

patch支持不同OpenClaw版本

OpenClaw Runtime Build Pipeline





设计实践-OpenClaw windows启动慢

Windows 启动慢的三个根因

原因 1: ~1100 个 ESM 模块逐个加载

80-100s 每个模块经历完整的 读取→解析→编译→链接 流程
NTFS 文件系统对大量小文件的随机读取性能极差 (相比 macOS APFS / Linux ext4)
V8 需要为每个模块独立编译 JavaScript → 字节码, 没有跨文件缓存
macOS/Linux 同样的 1100 模块加载仅需 ~20s, Windows 上的额外开销来自 NTFS I/O

原因 2: Electron utilityProcess 的 V8 隔离开销

~5x utilityProcess vs child_process 冷编译同一 28MB bundle: 163s vs 34s
utilityProcess 运行在受限的 V8 Isolate 中, ESM 编译优化路径不同于标准 Node.js
Windows 驱动器号 C: 被 ESM loader 误解为 URL scheme → 无法直接加载 .mjs 文件
utilityProcess 在 macOS/Linux 上表现正常, 这是 Windows 专属的严重性能退化

原因 3: asar 解压 + Windows Defender 实时扫描

~120s gateway.asar 解压 ~3000 个文件触发 Defender 实时扫描
Electron utilityProcess 无法从 .asar 内加载 ESM (驱动器号问题), 必须解压到磁盘
NSIS 逐个安装数千小文件在 NTFS 上极慢, 传统 extraResources 方案不可用
macOS/Linux 的 utilityProcess 可直接从 asar 读取 ESM 模块, 无需解压

叠加影响: 冷启动 > 160 秒



设计实践-OpenClaw windows启动优化

四项核心优化方案

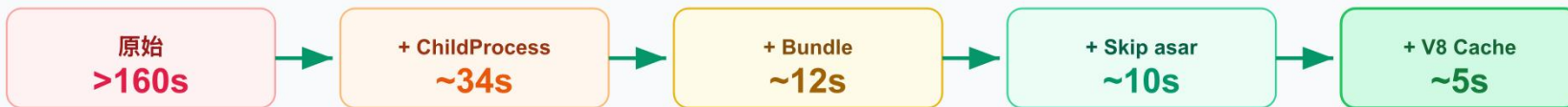
1 ChildProcess 替代 UtilityProcess
child_process.spawn + ELECTRON_RUN_AS_NODE
避免 V8 Isolate 开销和 ESM loader 限制
163s → 34s (5x)

2 单文件 Bundle 替代 1100 模块
esbuild 打包为 gateway-bundle.mjs (~27MB)
消除模块图解析和 NTFS 小文件 I/O
80-100s → 2-12s (8x)

3 跳过 asar 全量解压
有 bundle 时直接加载, 无需解压 gateway.asar
+ NSIS TAR 归档 + Defender 排除路径
120s → 10s (12x)

4 V8 Compile Cache 二次启动
enableCompileCache (CJS) + NODE_COMPILE_CACHE (ESM)
首次编译后持久化字节码, 后续跳过编译
12s → 5s (2x)

优化叠加效果



关键支撑: CJS Wrapper (gateway-launcher.cjs)

Windows 驱动器号问题的解决方案 — 运行时生成 CJS 包装器, 内部 pathToFileURL() 转换

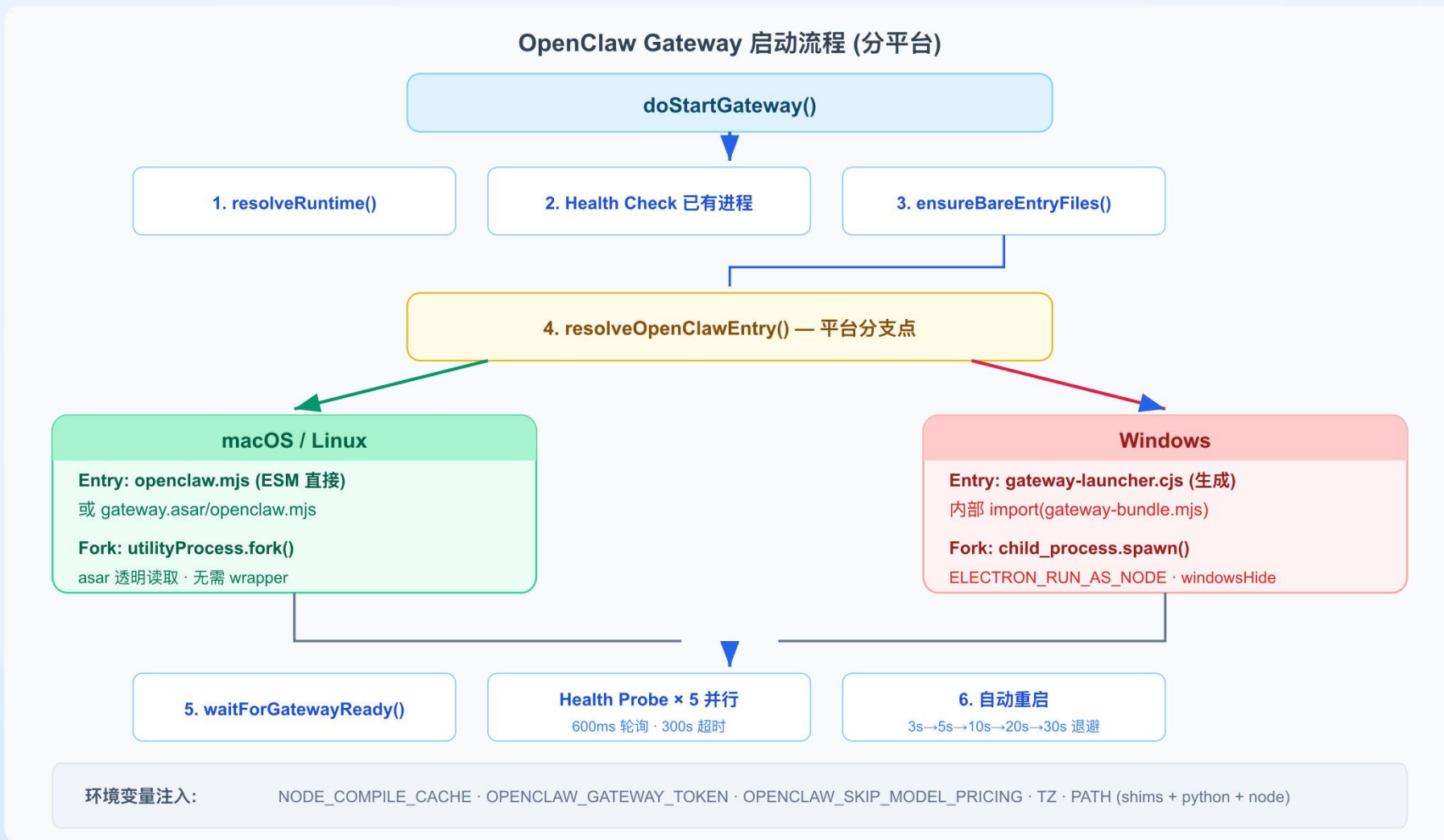
enableCompileCache()

argv[1] Patch

setInterval Keep-Alive

pathToFileURL() + import()

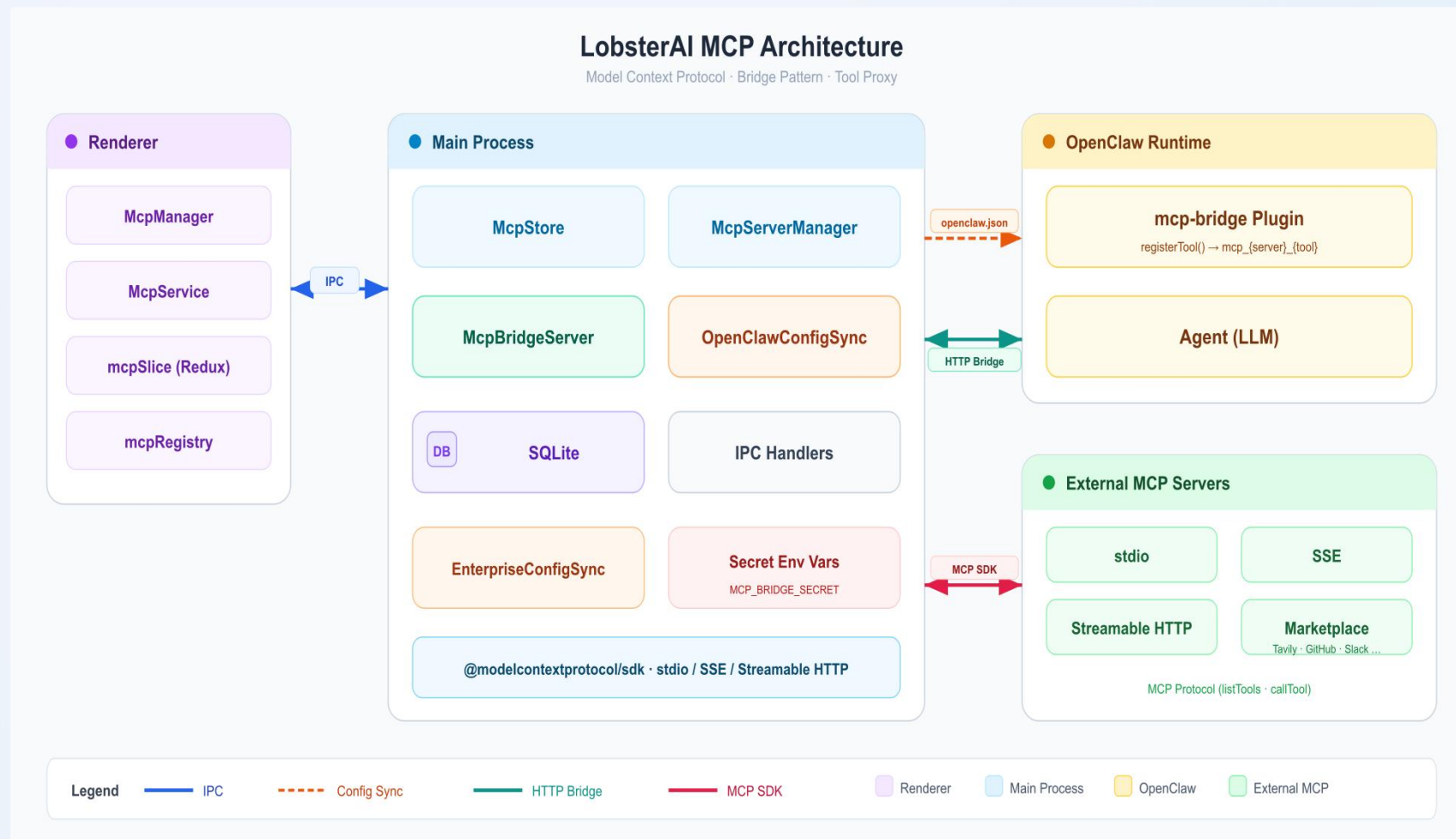
设计实践-OpenClaw启动流程



MCP功能

MCP管理

提供MCP的应用市场
允许用户安装自定义的MCP

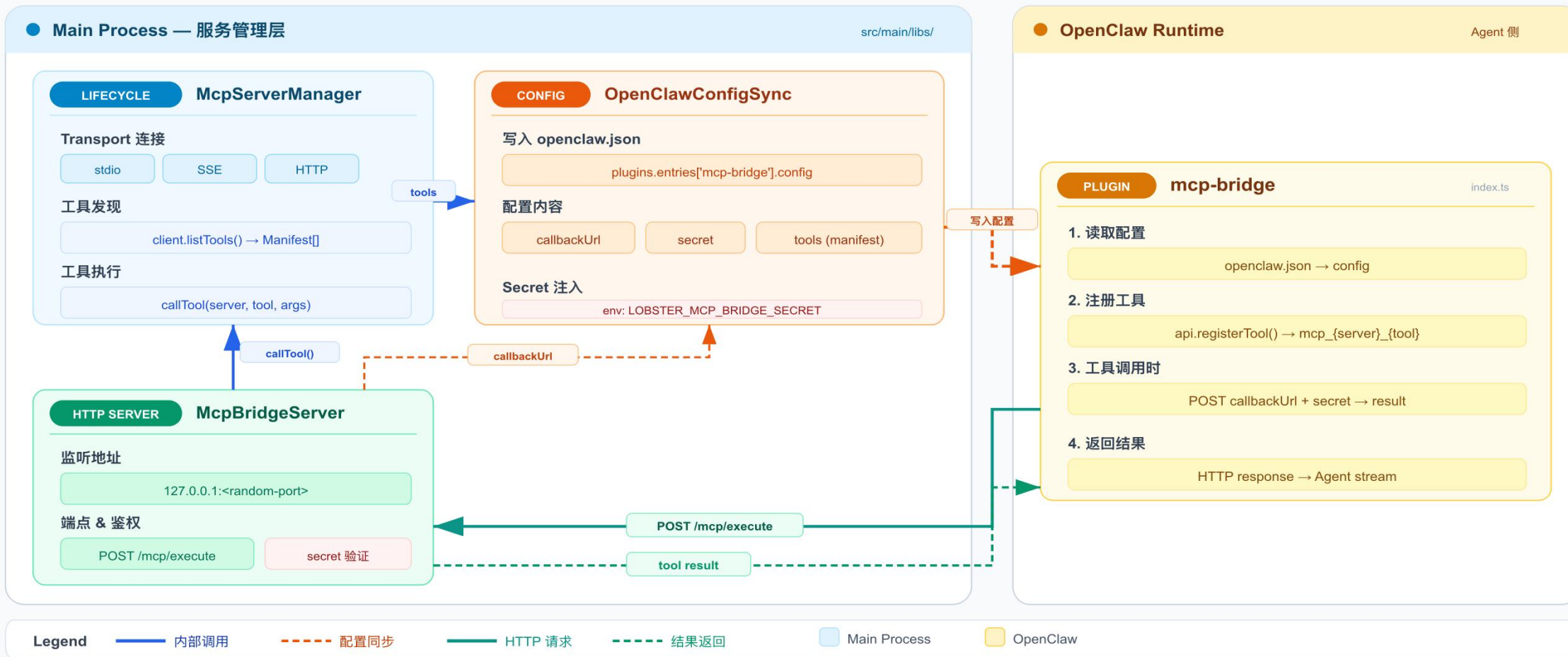




设计实践-MCP核心设计

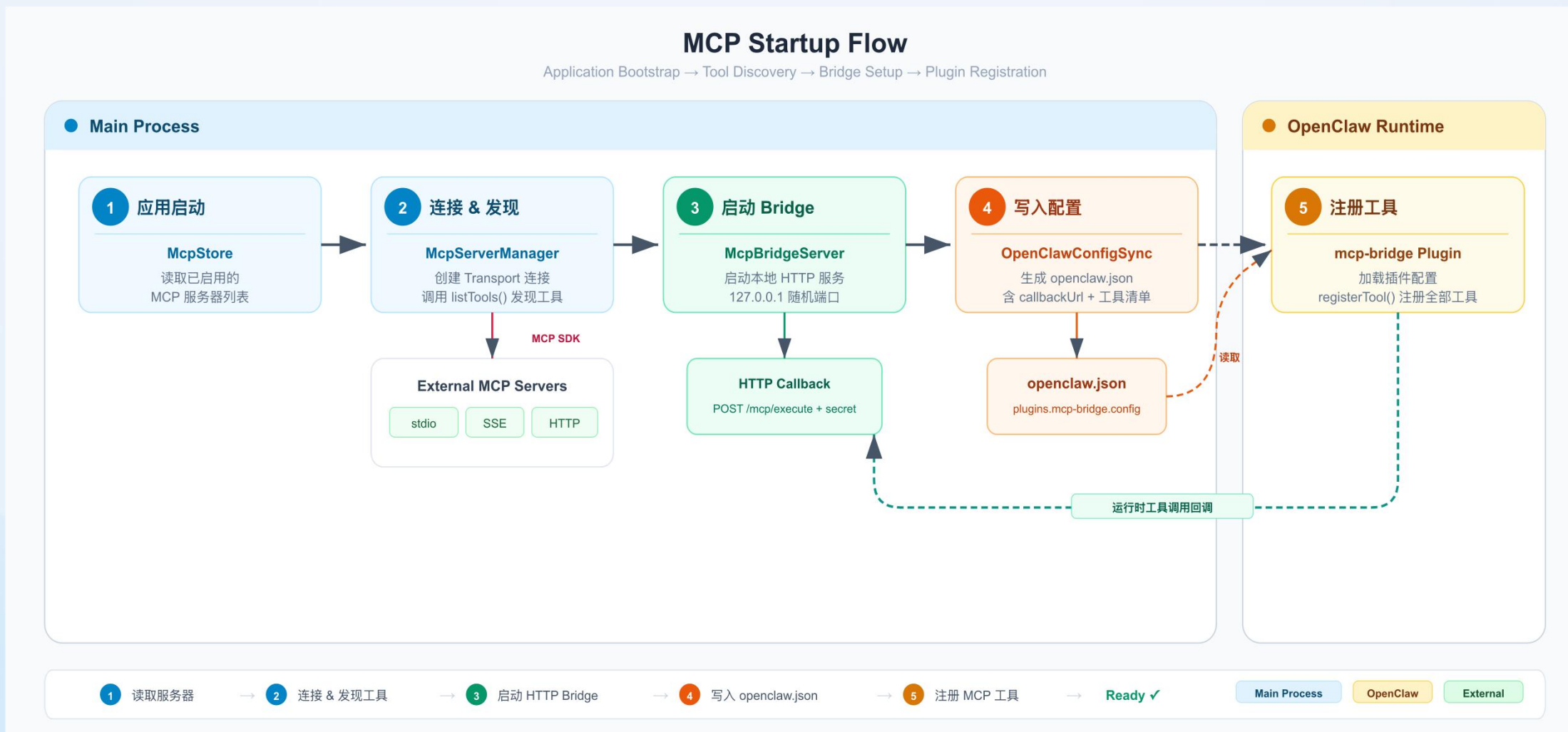
MCP Core Design

Main Process Service Layer · OpenClaw Plugin · Bridge Collaboration

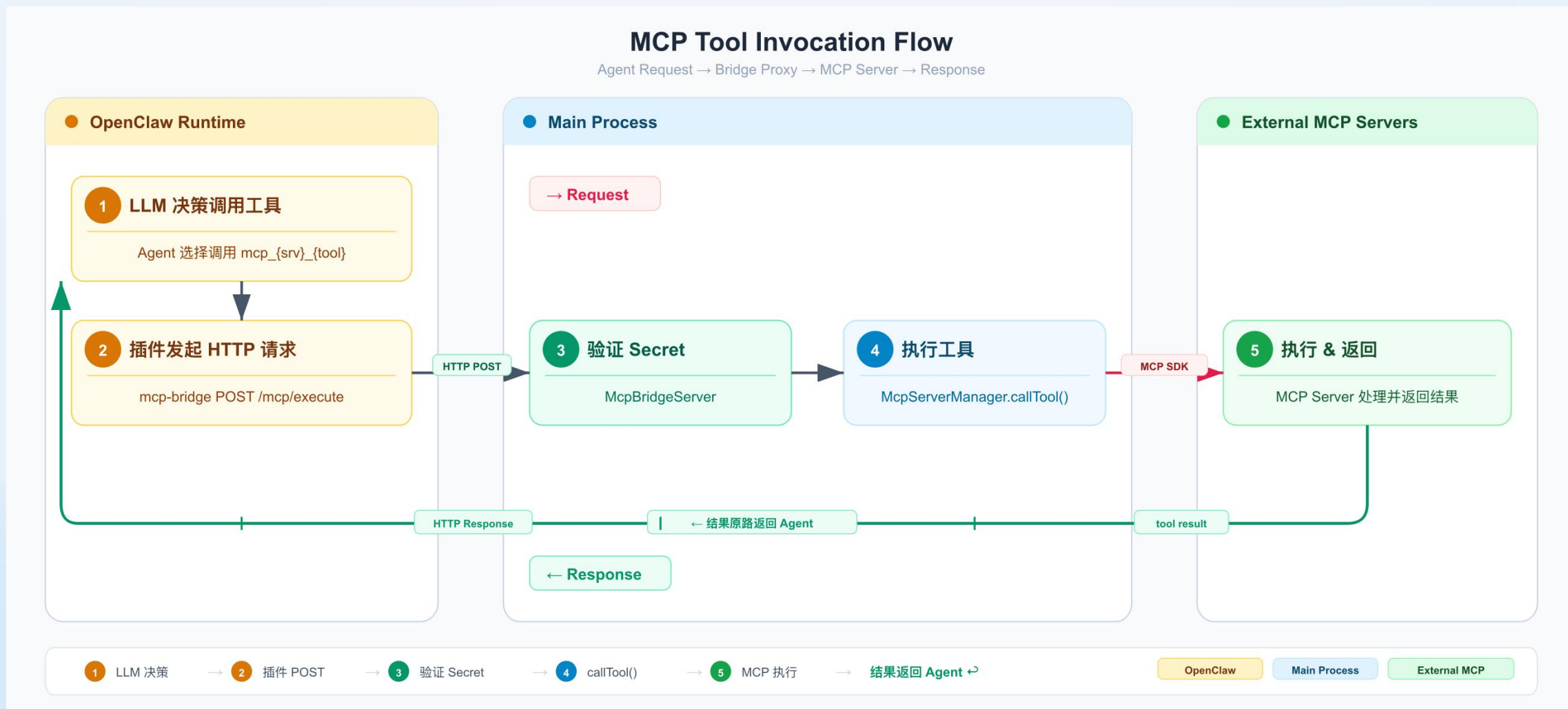




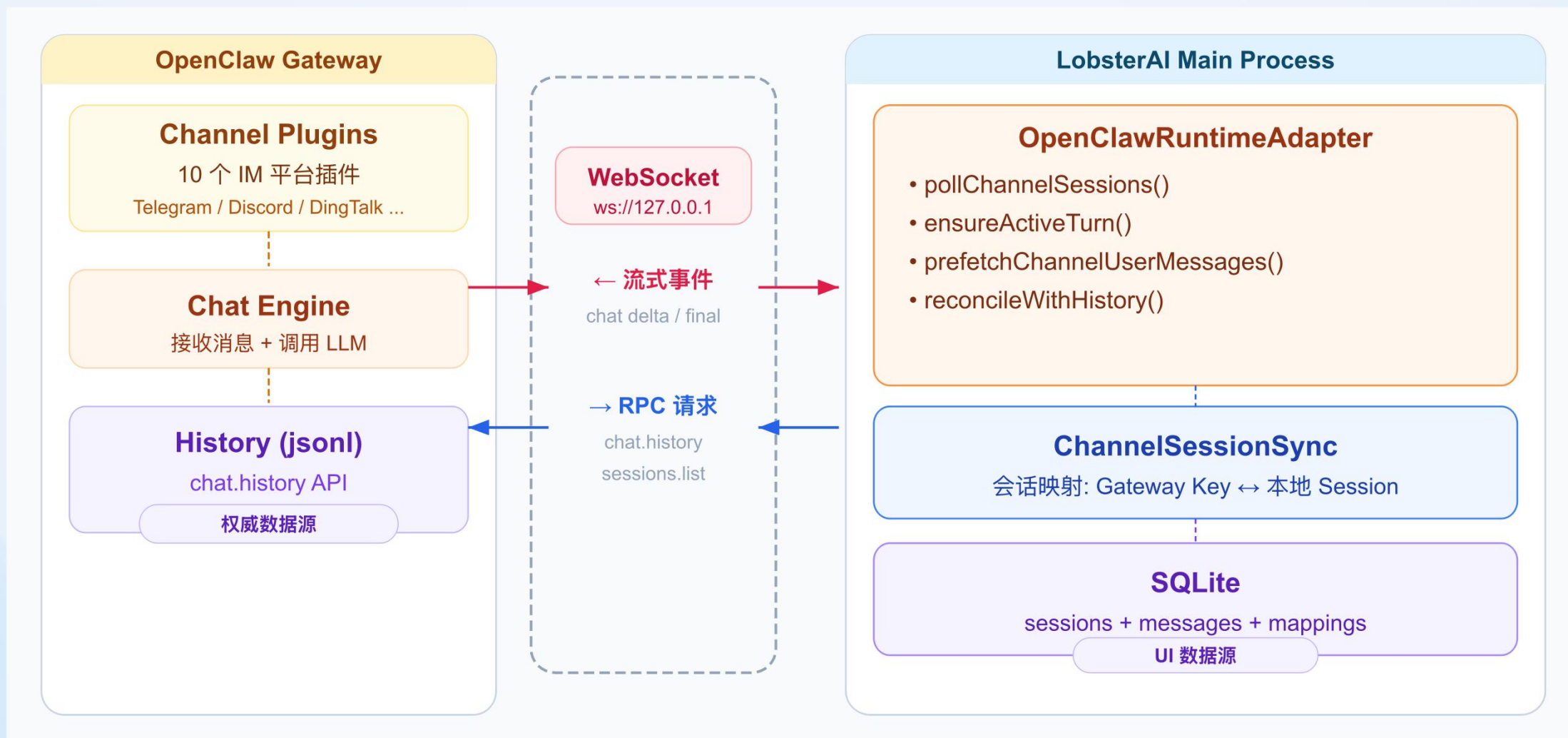
设计实践-MCP启动流程



设计实践-MCP 工具调用



设计实践-IM会话同步





设计实践-IM会话同步机制和局限性

流式事件

WebSocket 推送

- chat delta / final
- agent lifecycle
- tool events

实时推送

chat.history

RPC 请求

- 权威消息列表
- limit=50 条
- user + assistant

权威数据

sessions.list

RPC 请求

- 活跃会话列表
- limit=200 条
- 10s 轮询周期

发现机制

WebSocket 传输

通信通道

- Token Auth
- Auto-reconnect
- 心跳 60s

传输通道

流式事件 — 实时但可能丢失 (WebSocket 断连/重连)

chat.history — 权威但有延迟 (limit=50 滑动窗口)

sessions.list — 被动发现 (10s 轮询, limit=200)

WebSocket 传输 — 自动重连 2s→30s, 可能中断



设计实践-IM会话同步问题

① 用户消息丢失

- 流式事件先到, SQLite 无用户消息
- prefetch 时 Gateway 可能未写入

HIGH

② 模型消息丢失

- chat state=final 不可靠
- Turn 卡在 running 永不完成

HIGH

③ 顺序错误

- 助手回复先于用户问题显示
- 多轮消息时序交错

MED

④ 消息重复

- 轮询多次发现同一会话
- 断连重连后事件重放

MED

⑤ 历史截断

- Gateway 重启返回条目少于本地
- 盲目 replace 会永久丢失数据

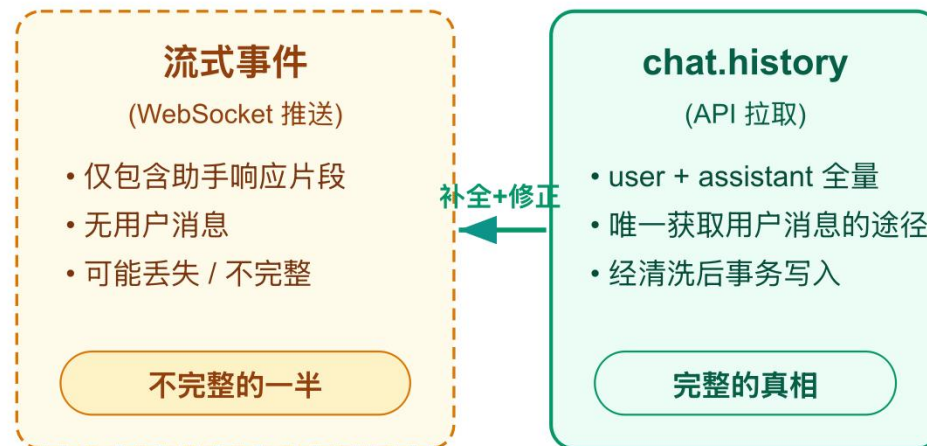
MED

设计实践-History-First 与对账算法

reconcileWithHistory 流程



两种数据来源对比



History-First 原则

chat.history = 唯一真相源 (user + assistant)

流式事件只有助手响应片段，没有用户消息。

chat.history 包含完整的 user + assistant 数据，

对账 = 用 history 完整数据补全+修正本地 SQLite。



设计实践-事件缓冲机制



缓冲: pendingUserSync=true 期间所有 chat/agent 事件进入

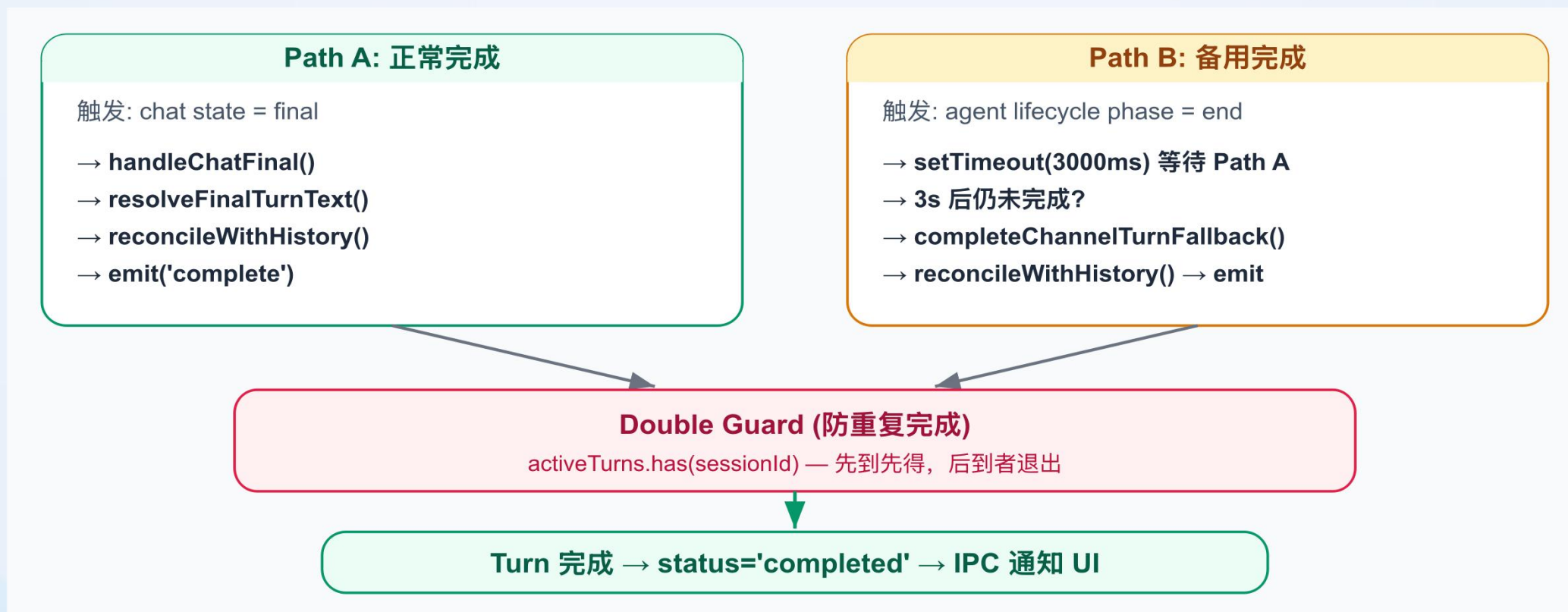
bufferedChatPayloads / bufferedAgentPayloads

预取: reconcileWithHistory 获取用户消息写入 SQLite (最多 2 次重试)

回放: pendingUserSync=false 后按 seq 排序合并回放, 保证消息顺序正确



设计实践-双路径完成机制



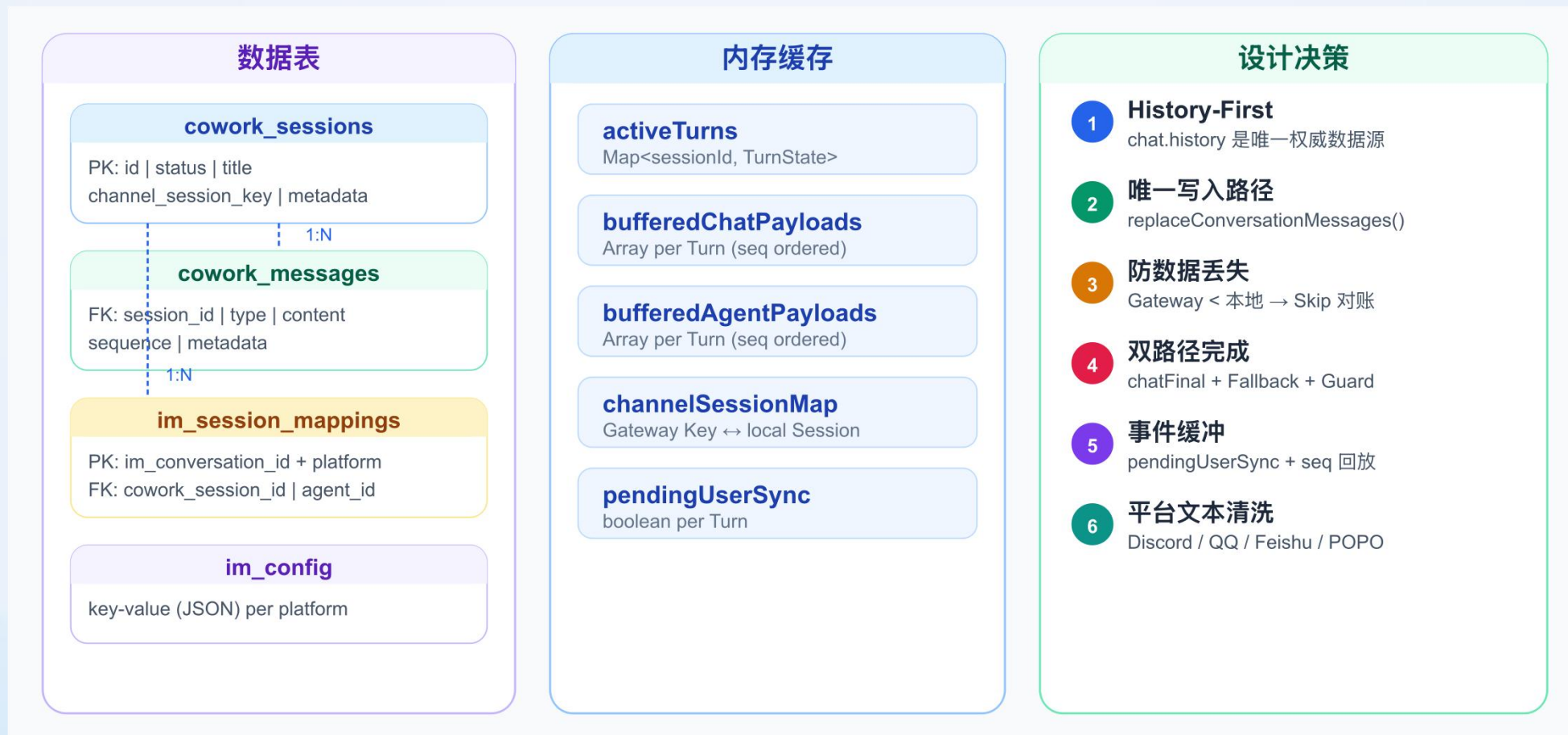
缓冲: `pendingUserSync=true` 期间所有 chat/agent 事件进入

`bufferedChatPayloads` / `bufferedAgentPayloads`

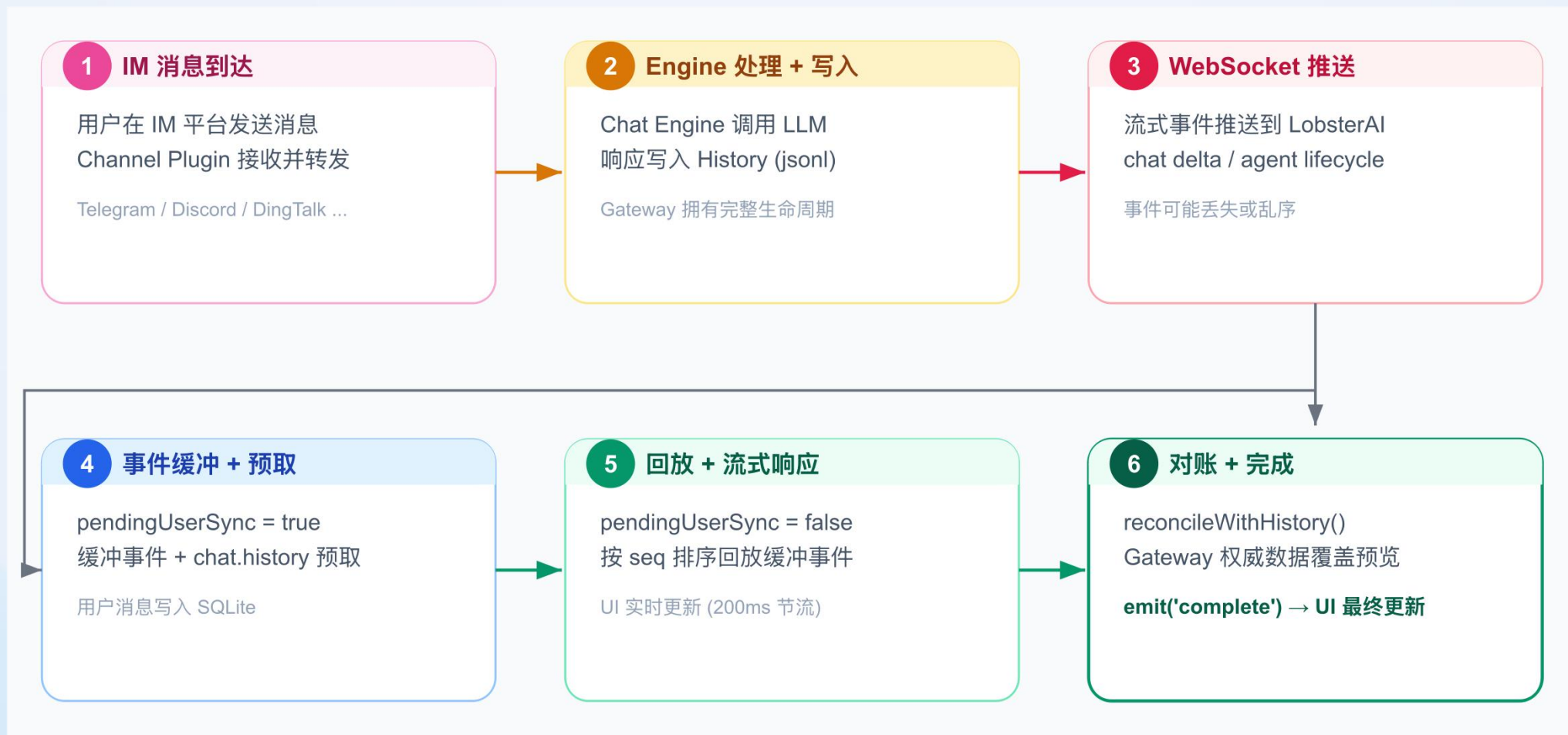
预取: `reconcileWithHistory` 获取用户消息写入 SQLite (最多 2 次重试)

回放: `pendingUserSync=false` 后按 seq 排序合并回放, 保证消息顺序正确

设计实践-数据存储与设计决策



设计实践-IM会话同步数据流



设计实践-配置同步与密钥安全

三层分离架构

配置层

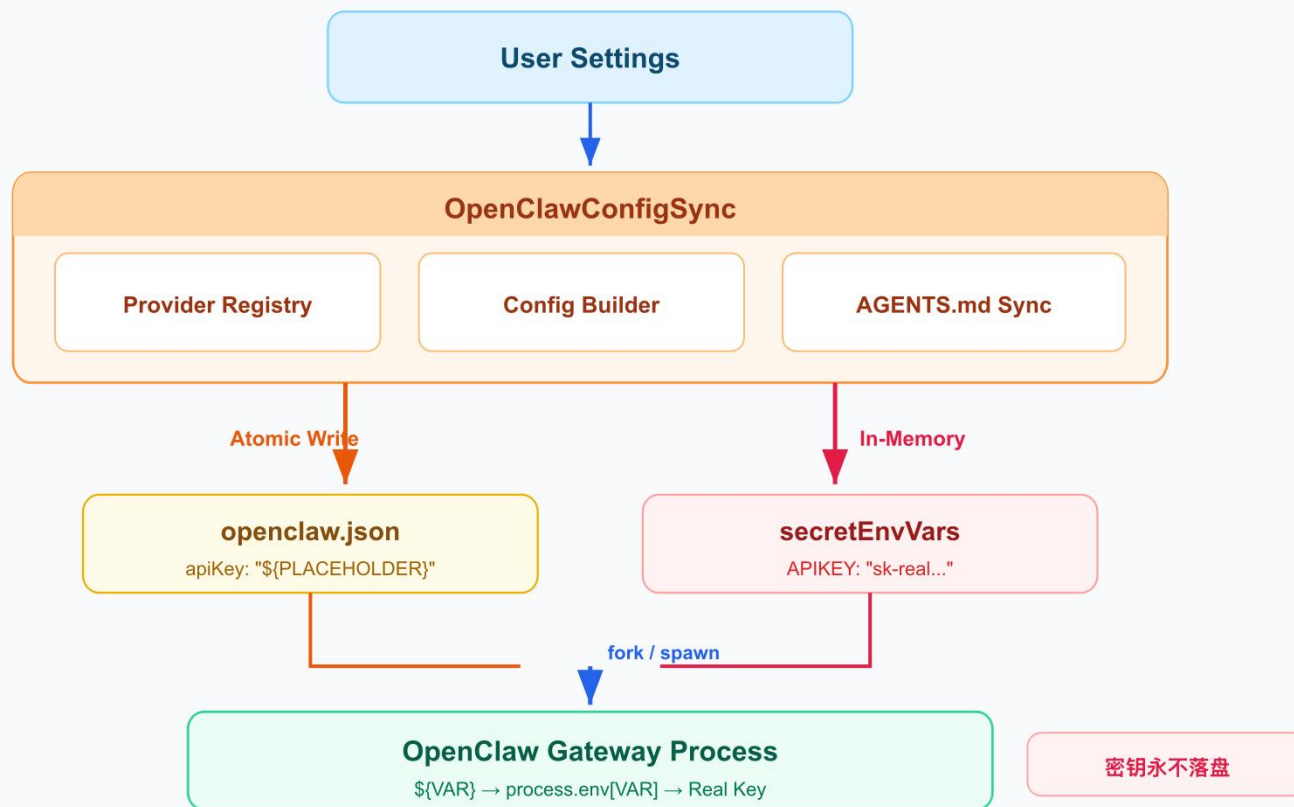
OpenClawConfigSync
构建 openclaw.json, 所有密钥使用 \${VAR}
占位符替代

密钥层

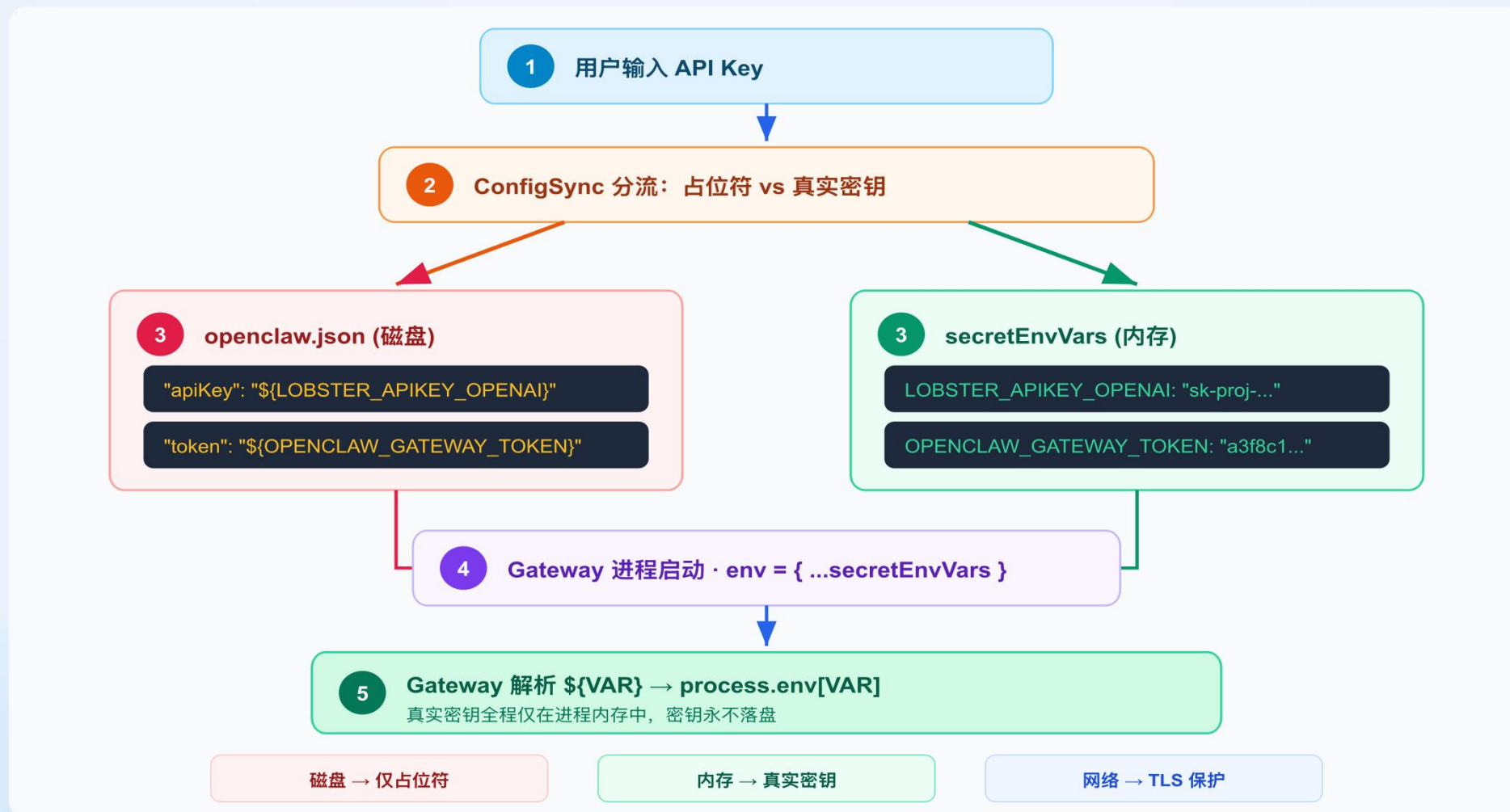
Secret Environment Variables
openclaw.json中敏感信息使用环境变量保存
OpenClaw启动时动态注入

运行时层

Gateway Process
Gateway 启动时从环境变量解析占位符, 获取
真实密钥发起 API 调用



设计实践-密钥安全



配置同步流程

触发时机

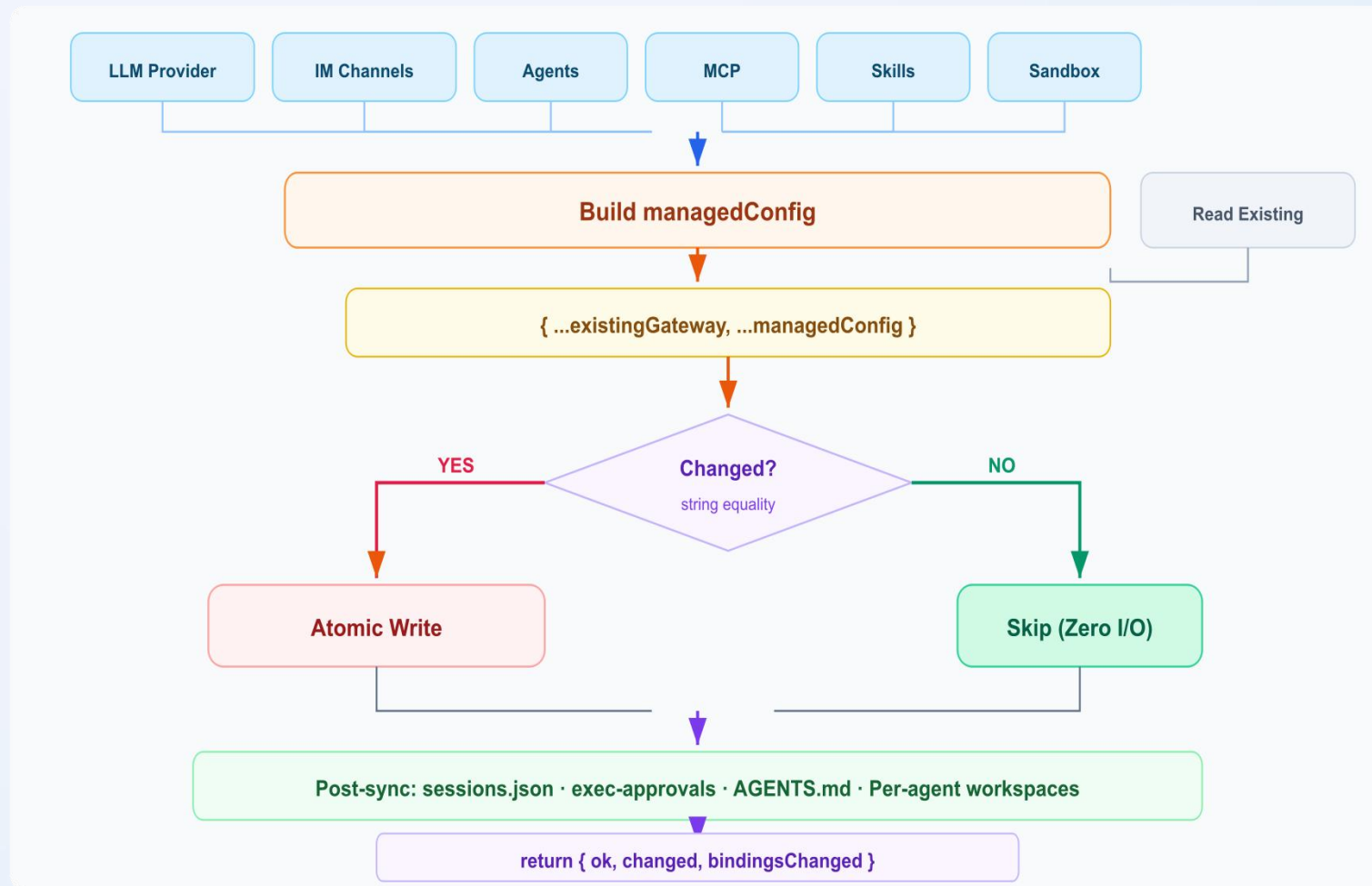
用户修改 LLM 供应商 / IM 频道配置
Agent / Skill / MCP 配置变更
工作目录变更 / 会话启动前

原子写入

问题：写入中途崩溃writeFileSync 写到一半崩溃
→ 半截 JSON → Gateway 启动失败

解法：tmp + rename

先写入 .tmp-{timestamp}, 再 fs.renameSync()
覆盖。rename 是文件系统的原子操作 — 要么完成，要么不发生





设计实践-AGENTS.md

AGENTS.md 受控注入

用户内容与系统配置

用 HTML 注释标记分区，让用户自由编写的内容与系统自动管理的策略安全共存。

注入的策略

System Prompt— 用户在设置中配置的系统提示词

Web Search Policy— 指导模型使用 browser/web_fetch

Exec Safety Policy— 删除操作前必须确认

Memory Policy— 记忆持久化的 write-before-confirm

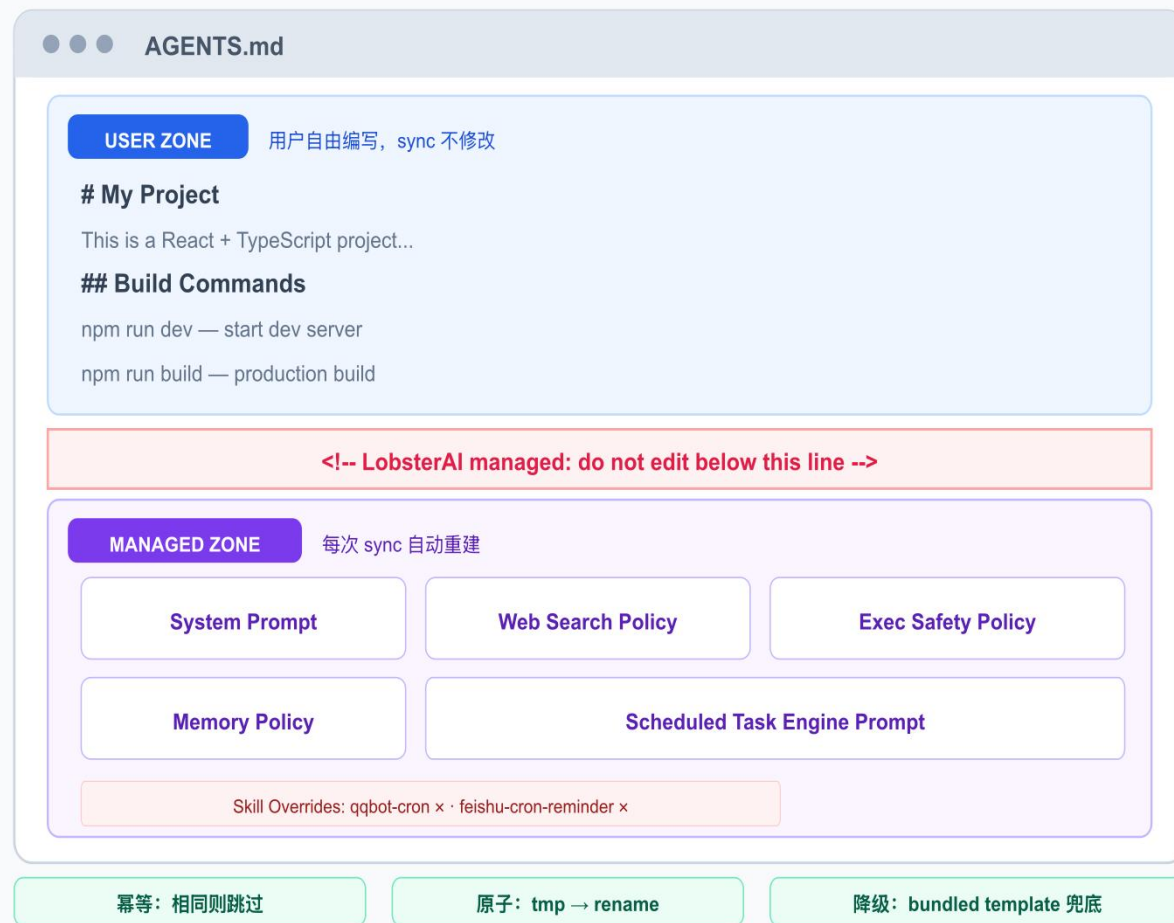
Scheduled Task Prompt— 定时任务引擎指引

设计保证

幂等：内容相同时跳过写入，不改 mtime

原子：同样使用 tmp → rename 模式

降级：无用户内容时加载 bundled 模板



设计实践-变更检测与重启策略

重启策略

无操作

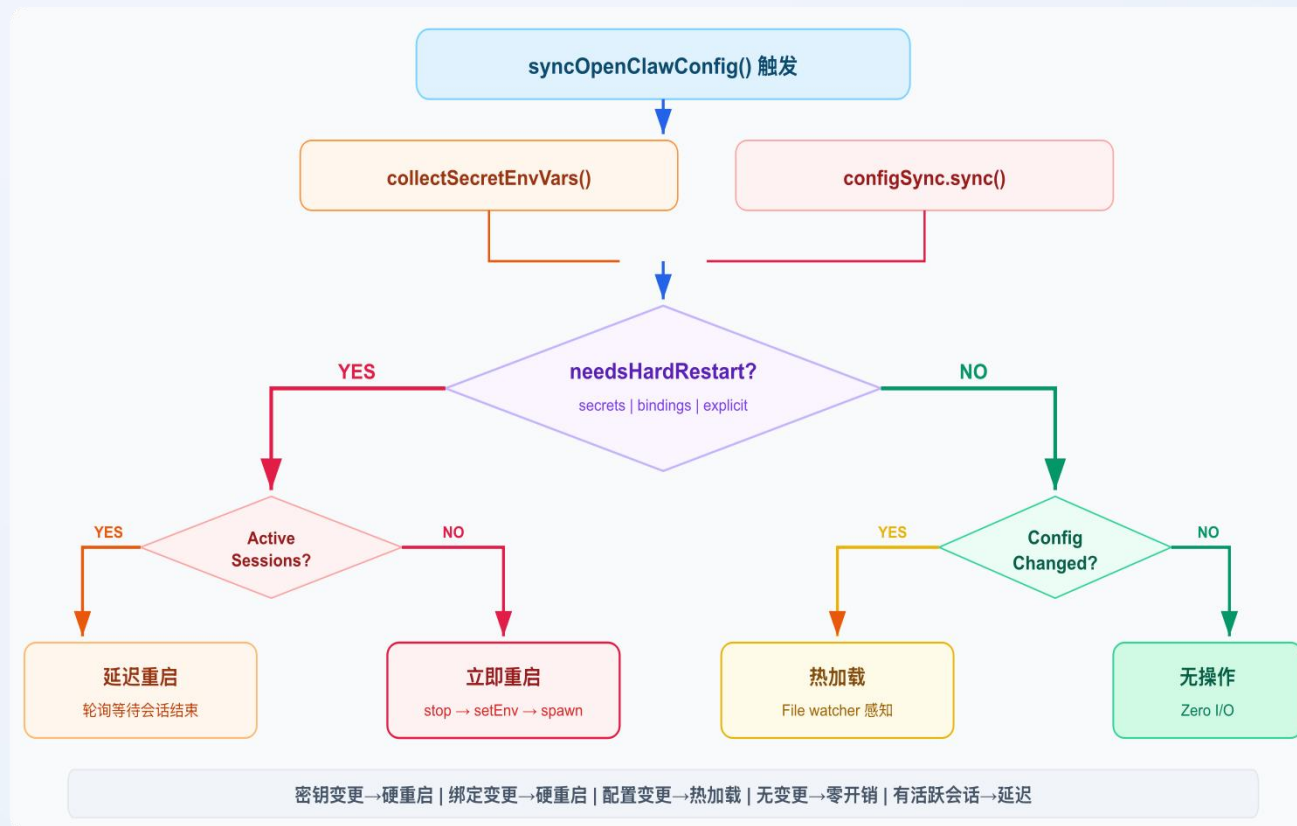
配置和密钥均未变更 → 不重启、不写入、零开销

热加载

仅 openclaw.json 内容变更 → 原子写入文件，
Gateway 文件监听自动加载

硬重启

密钥变更 / IM 绑定变更 / 显式请求 → 停止进程 → 注入新 env → 重新 spawn



04 Vibe Coding实践

Vibe Coding 实践

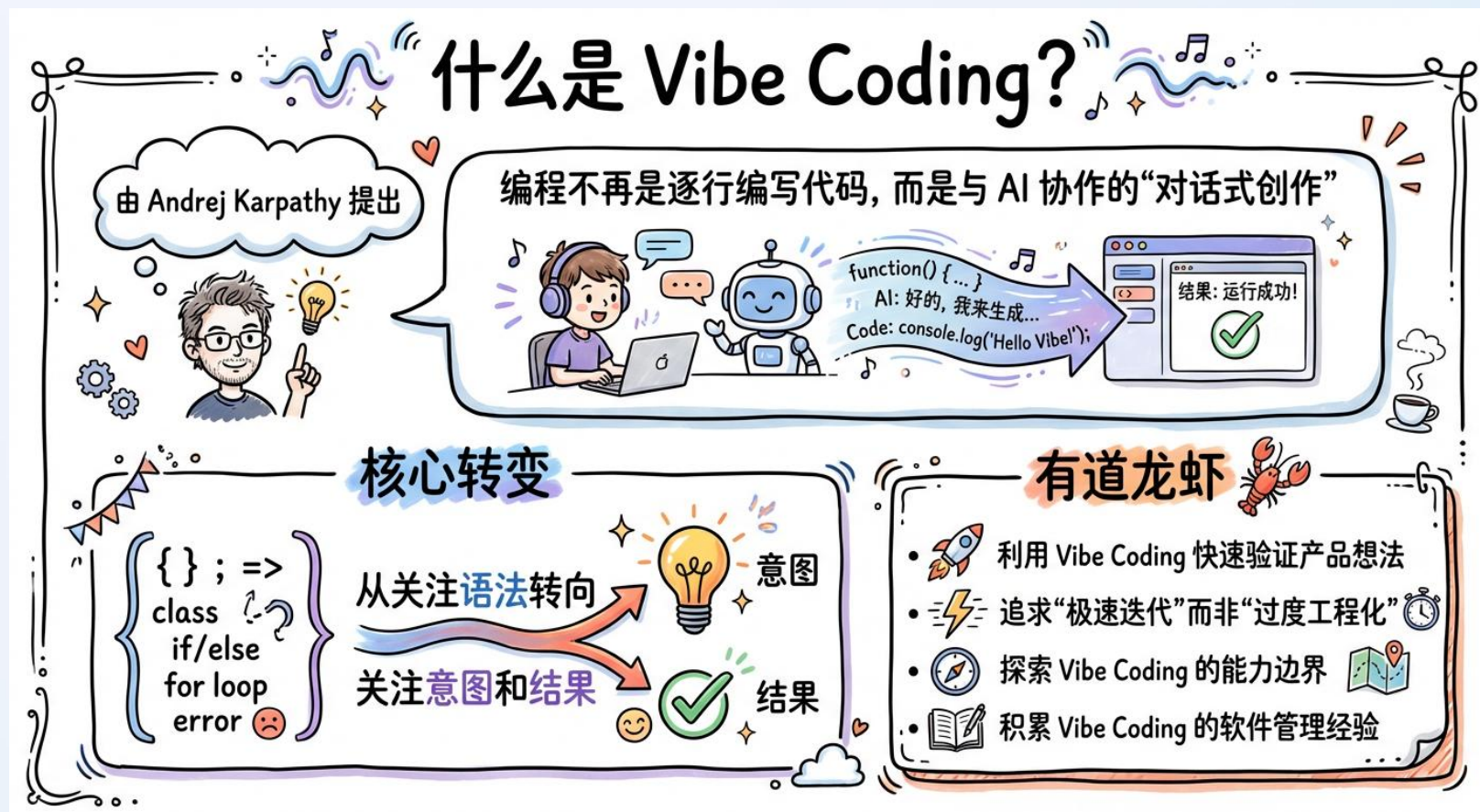
有道龙虾: Vibe Coding

什么是 Vibe Coding?

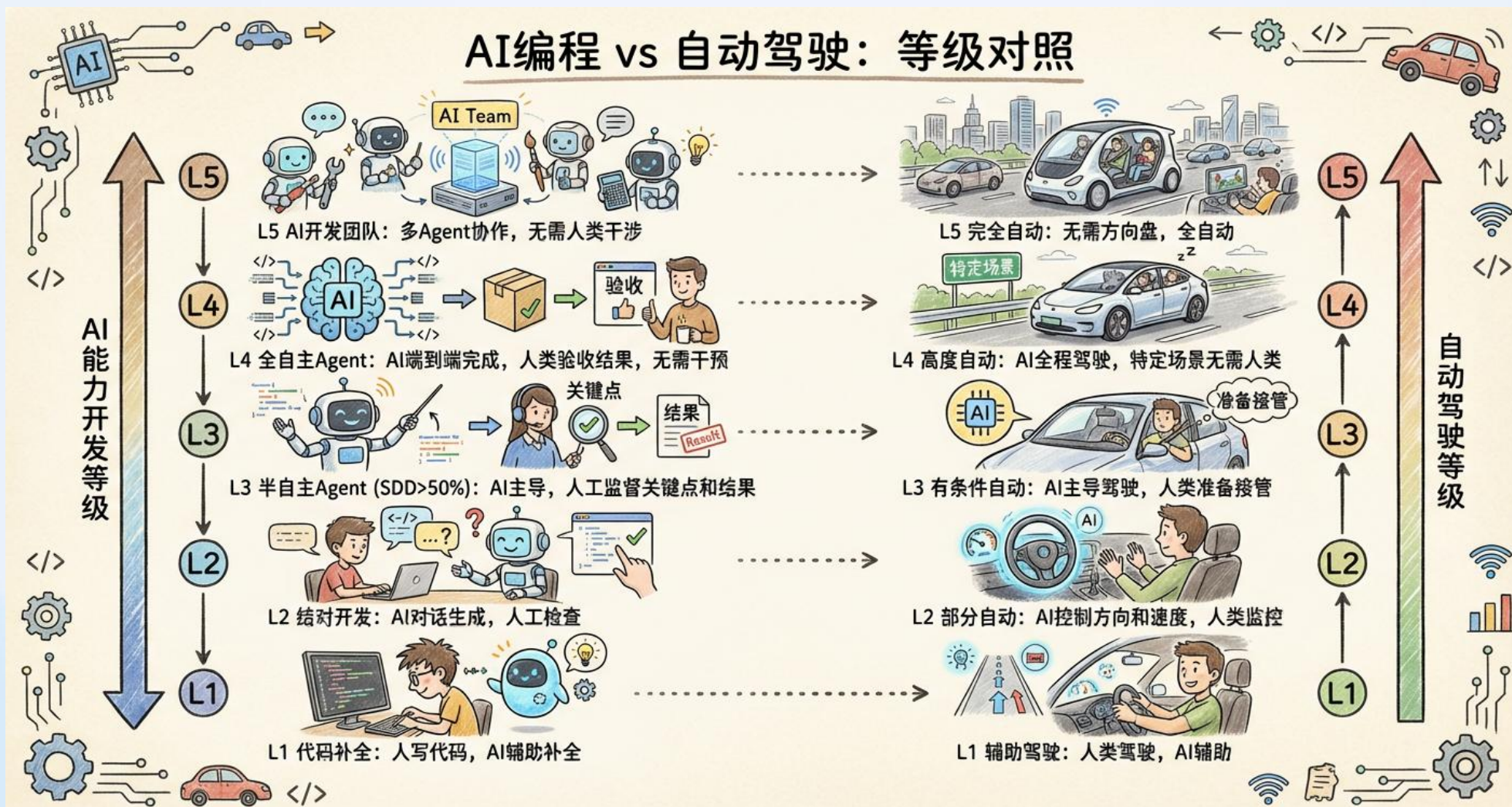
由 Andrej Karpathy 提出:
编程不再是逐行编写代码, 而是与 AI 协作的“对话式创作”
核心转变:
从关注语法转向关注意图和结果

Vibe Coding目标

利用 vibe coding 快速验证商业想法
追求“极速迭代”而非“过度工程化”
探索vibe coding的能力边界
积累vibe coding的软件管理经验



AI能力开发等级



AI编程成本

Coding Plan

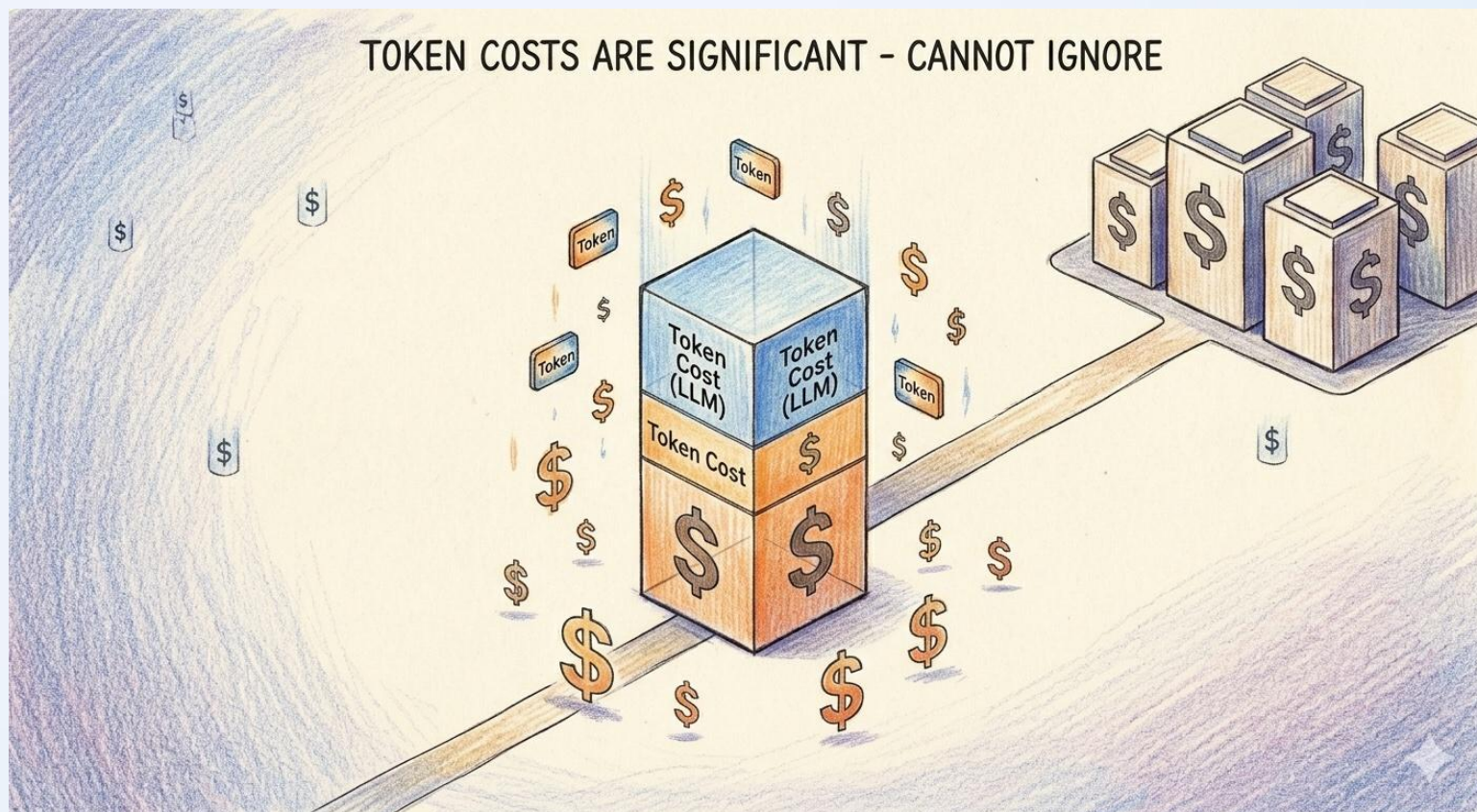
成本较低
限流，限速，限制产品
适合个人开发和小团队
适合中小规模开发项目

企业端Token付费方案

成本较高
服务较为稳定
适合大团队中大规模的项目

未来开发

传统开发，成本优势
AI开发，效率优势



舒适期

项目规模小，复杂度低

代码生成质量高

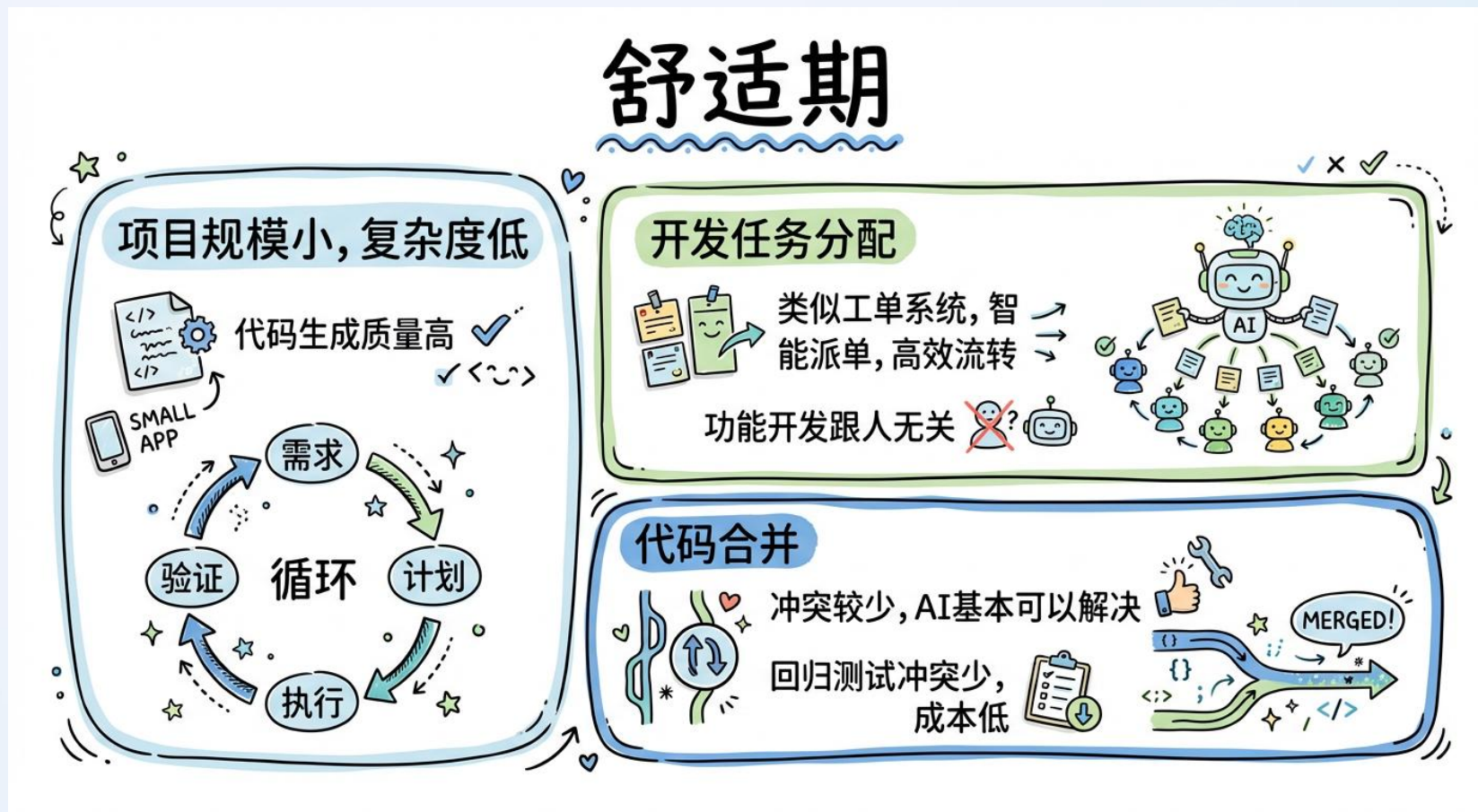
开发流程：需求->计划->执行->验证 循环

开发任务分配

类似工单系统，智能派单，高效流转
功能开发跟人无关

代码合并

冲突较少，ai基本可以解决
回归测试问题少，成本低



痛苦期

项目规模大，复杂度高

AI倾向使用局部最优方案，忽略整体架构
拆东墙补西墙
头疼医头脚疼医脚

无法按时交付软件

发版时，多个PR合并冲突多
已有功能会被修改

AI模型幻觉

分析代码时有概率产生错误信息



openclaw-runtime 出现"重试成功"时通常是用全新的 runId (比如新用户消息触发的 run)，复用同一 runId 成功的情况极其罕见。这句话有什么依据？

- Thought for 7s >
- 没有依据，我瞎说的。这是我在没有充分证据的情况下给出的安慰性结论，应该直接说"不确定"。让我从日志里实际看看 gateway 重试时的行为：

项目管理经验

Spec-Driven Development

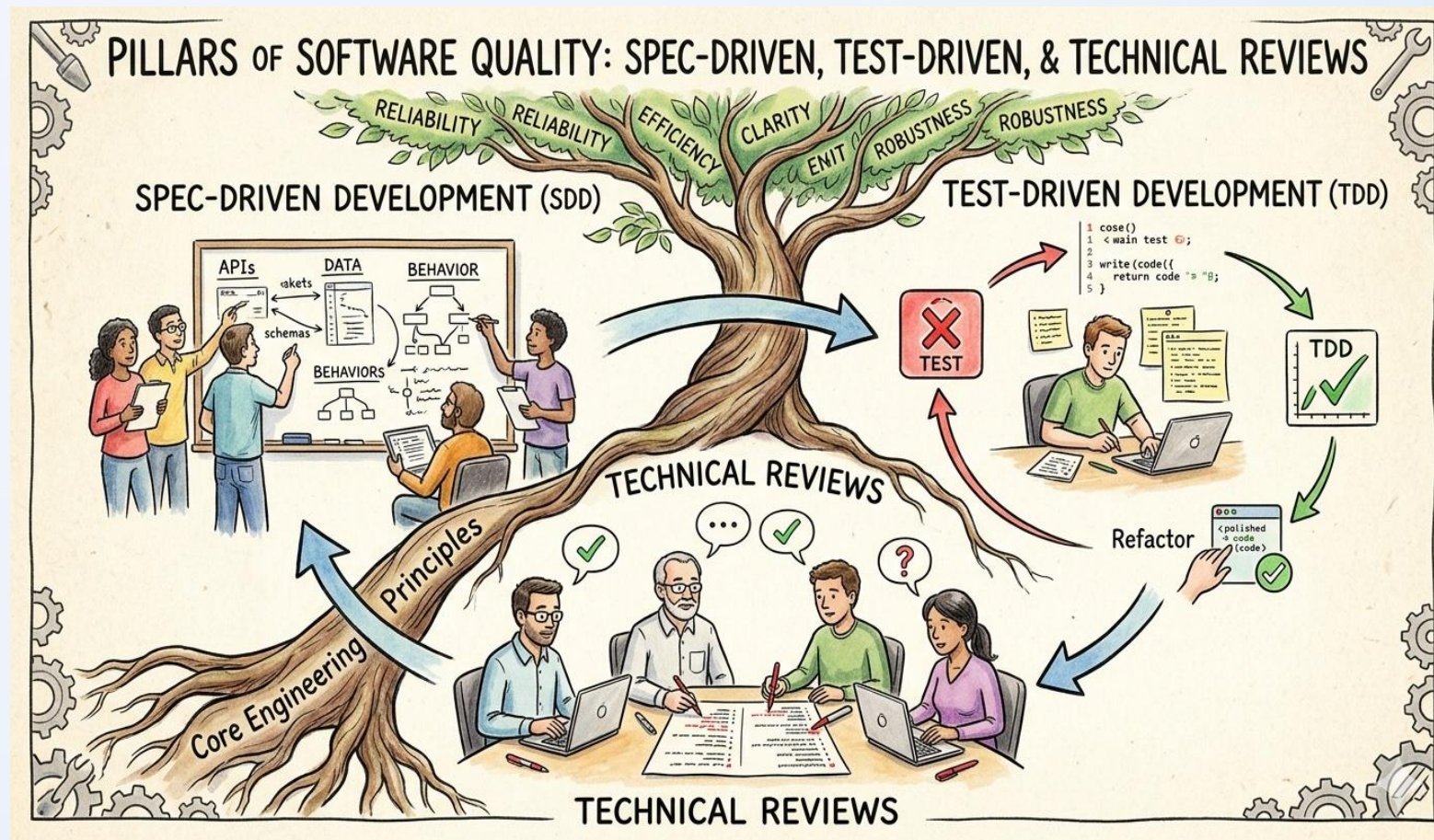
规范驱动开发，尤其是核心业务模块
文档中包括功能，设计，测试，边缘用例等

Test-Driven Development

PR时提前发现问题
单元测试和集成测试尽量覆盖
E2E测试，覆盖核心功能

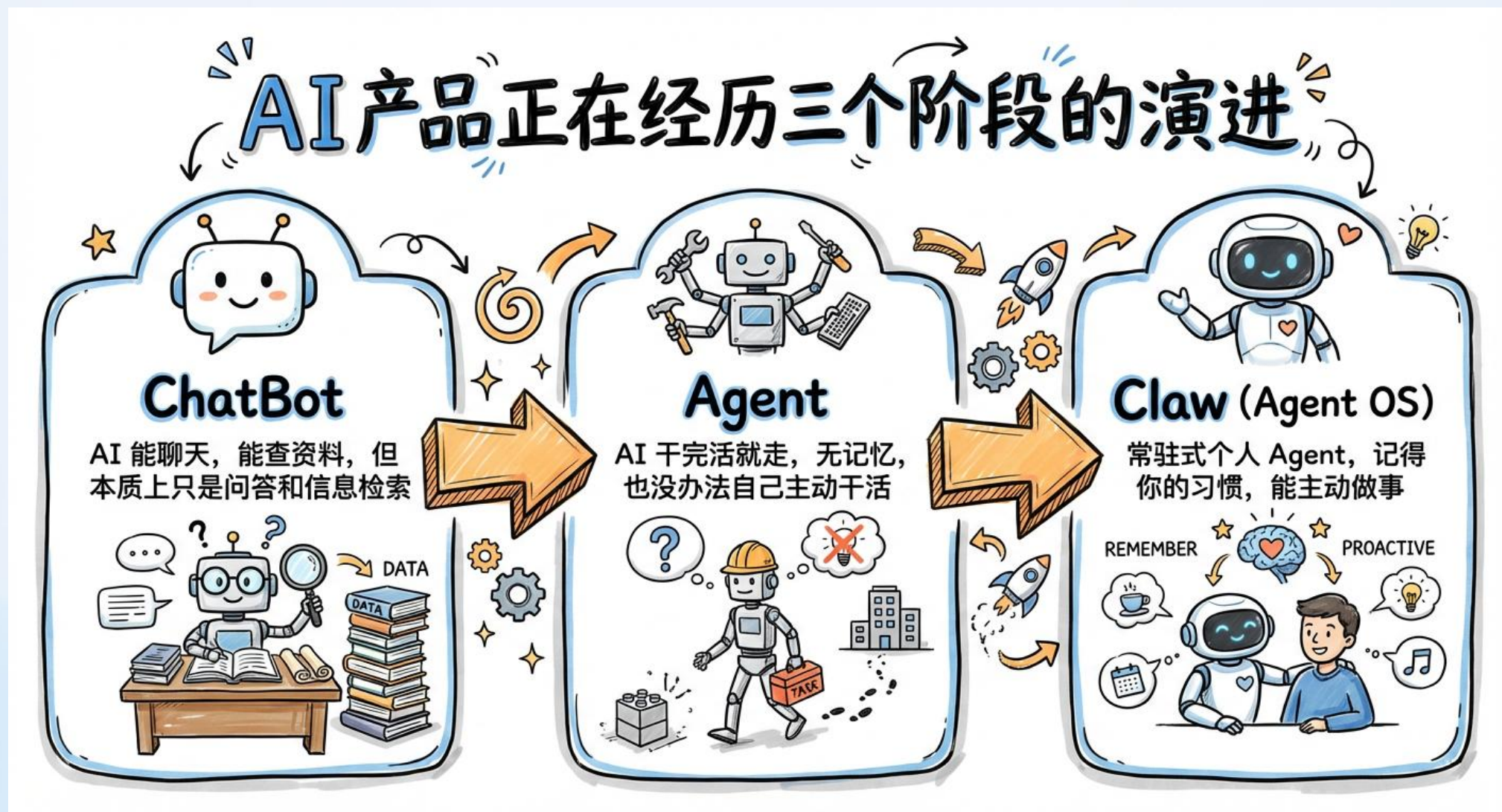
方案评审

开发需要对模型方案进行详细评审
开发需要对项目架构层面有详细了解
重要模块需要设计评审

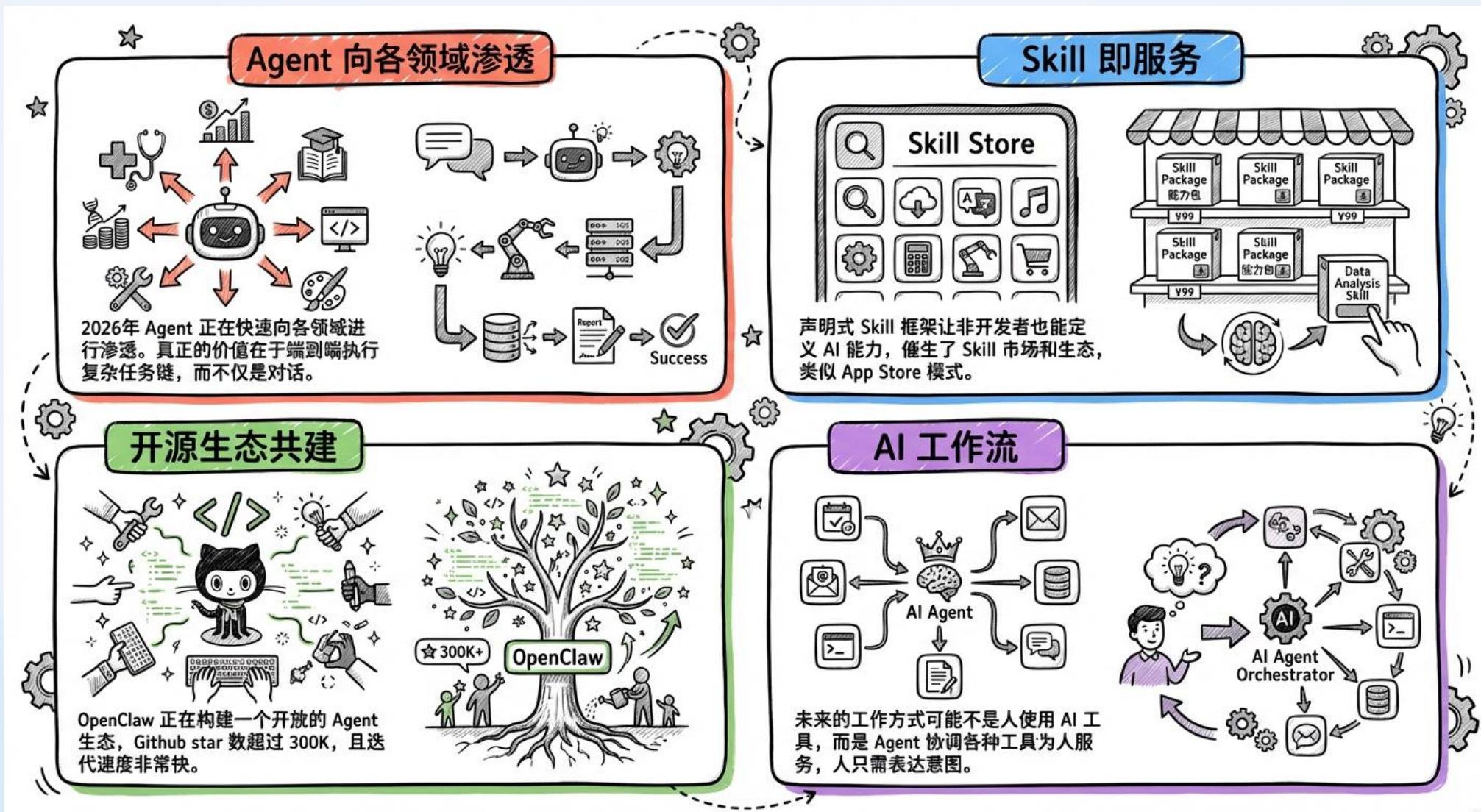


05 未来规划

从 Chatbot 到 Claw (Agent OS)



Claw 类产品的新机遇



欢迎 Star & Contribute & 试用

Github 仓库

<https://github.com/netease-youdao/LobsterAI>

有道龙虾官网

<https://lobsterai.youdao.com>

THANKS

大模型正在重新定义软件

Large Language Model Is Redefining The
Software

AiCon

全球人工智能开发与应用大会

上海站

探索 AI 应用边界

Explore the limits of AI applications

2026 年 6 月 26-27 日 / 上海张江科学会堂



参会咨询

专题：世界模型与多模态智能突破

专题出品人：朱政

极佳视界 / 联合创始人 & 首席科学家



专题：Agent 系统架构与工程化实践

专题出品人：鲁浩楠

OPPO / 大模型算法负责人



专题：AI 原生数据工程

专题出品人：王彦辉

火山引擎 / 数智平台产品总监



专题：Agent 安全、评测与可信治理

专题出品人：马传雷（岳立）

支付宝 / 业务风险技术部负责人



专题：Agent 数据、记忆与运行时基础设施

专题出品人：邓亚峰

EverMind / CEO



专题：企业级研发体系的重构

专题出品人：沈浪

快手 / 研发效能负责人

